

<https://doi.org/10.1038/s41524-024-01250-5>

Physics and chemistry from parsimonious representations: image analysis via invariant variational autoencoders

Mani Valleti¹✉, Maxim Ziatdinov², Yongtao Liu³ & Sergei V. Kalinin^{2,4}✉

Electron, optical, and scanning probe microscopy methods are generating ever increasing volume of image data containing information on atomic and mesoscale structures and functionalities. This necessitates the development of the machine learning methods for discovery of physical and chemical phenomena from the data, such as manifestations of symmetry breaking phenomena in electron and scanning tunneling microscopy images, or variability of the nanoparticles. Variational autoencoders (VAEs) are emerging as a powerful paradigm for the unsupervised data analysis, allowing to disentangle the factors of variability and discover optimal parsimonious representation. Here, we summarize recent developments in VAEs, covering the basic principles and intuition behind the VAEs. The invariant VAEs are introduced as an approach to accommodate scale and translation invariances present in imaging data and separate known factors of variations from the ones to be discovered. We further describe the opportunities enabled by the control over VAE architecture, including conditional, semi-supervised, and joint VAEs. Several case studies of VAE applications for toy models and experimental datasets in Scanning Transmission Electron Microscopy are discussed, emphasizing the deep connection between VAE and basic physical principles. Python codes and datasets discussed in this article are available at <https://github.com/saimani5/VAE-tutorials> and can be used by researchers as an application guide when applying these to their own datasets.

Since the invention of the optical microscope by Levenhoek¹, microscopy has played progressively growing role in science. Modern optical microscopy is the mainstream characterization tool in biology², materials sciences^{3–6}, nanotechnology⁷, and spans desktop tools to bespoke confocal and light sheet imaging systems. Subsequently, scanning probe microscopy has literally opened the door to the nanoworld⁸, and enabled visualization of structures as well as electronic^{9–11}, magnetic^{12,13}, chemical¹⁴, and electromechanical¹⁵ functionalities. Electron microscopy has become the mainstay of materials science¹⁶, condensed matter physics¹⁷, molecular biology, chemistry and catalysis¹⁸.

Common for all areas of electron, optical, and probe microscopy imaging is the progressive transition from the qualitative imaging when only the salient features of image are qualitatively explored to quantitative imaging. In optical imaging, this is exemplified by the multitude of the biological imaging modalities, metallographic imaging, and broad variety of hyperspectral imaging modes including microRaman and photoluminescence

measurements. In electron and scanning probe microscopy, the examples include mapping order parameter fields¹⁹, learning the internal structures of grain boundaries²⁰, and decoding complex structures and defect chemistries of 2D materials. For scanning probe microscopy, this includes techniques such as molecular unfolding spectroscopy^{21,22} that yields quantitative mechanisms of single-molecule reactions and piezoresponse force microscopy²³ that quantifies ferroelectric polarization dynamics.

Over the last decade, the volumes of data acquired over the modern imaging tools have grown exponentially, and now represent large images, multimodal images comprising multiple channels, hyperspectral datasets, and dynamic images such as videos. This transition to the quantitative imaging was largely enabled by the introduction of the computational systems for storing and analysis of the data on the level of the individual tools, facilities, and inter-facilities networks. At this point, virtually all microscope systems are equipped with the local data storage and analysis capabilities. While this transition is almost complete now, it is important to

¹Bredesen Center for Interdisciplinary Research, University of Tennessee, Knoxville, TN, 37916, USA. ²Physical Sciences Division, Pacific Northwest National Laboratory, Richland, WA, 99352, USA. ³Center for Nanophase Materials Sciences, Oak Ridge National Laboratory, Oak Ridge, TN, 37831, USA. ⁴Institute for Advanced Materials and Manufacturing, Department of Materials Science and Engineering, University of Tennessee, Knoxville, TN, 37996, USA.

✉ e-mail: svalleti@vols.utk.edu; sergei2@utk.edu

note that this trend is relatively recent and is largely confined to the last 20 years. The emergence of the Google Colabs and jupyter notebooks over the past 5 years have enable simple code and analysis workflow sharing between the individual researchers. Finally, the rapid growth of the cloud capabilities such as Amazon AWS, Google Cloud, or Microsoft Azure now opens the pathway for integration of data flows between multiple facilities and cloud-based storage and analytics of the imaging data, where only the analysis results are visualized on the personal devices.

The availability of these capabilities and the datasets that contain information about local structures functionality, and dynamics, necessitate the development of the data analysis tools for extracting the salient features of system behavior. Traditionally, such analyses have been based on the classical computer vision algorithms including local and semi local filters, Fourier, Radon, and Hough transforms²⁴, etc. requiring extensive manual development and often elegant, but highly personalized, workflows. The emergence of the supervised deep learning methods in images²⁵ has offered an alternative approach and rendered many traditionally complex problems such as semantic segmentation, instance segmentation, etc. straightforward. This was done simply by translating workflows from computer and medical imaging to the microscopy domains. The advancements in the ensemble methods^{26–30} have also allowed to address the out of distribution drift effects and opened the pathways to engender these methods as a part of real time microscope operation²⁹.

However, the supervised methods, by definition, apply only if the physical picture of the phenomena of interest is known. Practically, in many applications scientists seek to discover unknown physical phenomena such as manifestations of symmetry breaking phenomena in electron and scanning tunneling microscopy images, or variability of the nanoparticles, etc. These problems constitute the unsupervised physical discovery problems and necessitate the development of corresponding algorithms. These in turn should satisfy the practical considerations present in the analysis of imaging data, i.e., address the imaging system distortion, rotational invariances in image plane, allow for the fully unsupervised and semi-supervised learning tasks, and allow for combined clustering and elucidation of factors of variance.

Here, we review the principles and applications of the Variational Autoencoders (VAEs), a powerful method for the analysis of high-dimensional data. We discuss the basic principles and intuition behind VAE using the simple toy datasets and introduce real-world examples in supervised, semi-supervised and unsupervised settings. We specifically emphasize the links between VAEs and physics of the system. We further discuss invariant autoencoders, both to discover the salient aspects of materials structure and allow for the distortions inevitable in real imaging systems. Finally, we discuss the VAE architectures including conditional, joint, and semi-supervised VAEs, and discuss the types of the image analysis tasks they can be applied to. All these methods are incorporated as a part of 'VAE-tutorials' GitHub repository and is available for public access at <https://github.com/saimani5/VAE-tutorials>.

Results and discussions

Variational autoencoders have been introduced by Kingma^{31,32} and Rezende³³ almost simultaneously in 2014 and are a special class of generative probabilistic models. They possess multiple distinctive properties that make them singularly useful for image analysis, and especially physics-based analytics. Below, we discuss the basic principles of VAEs and compare them to classical autoencoders, briefly introduce mathematical formalism beyond VAEs, and illustrate their behavior via., simple examples. Here and throughout the article, we would like to highlight that, as for many machine learning (ML) tools, application of VAEs to real world problems requires development of both general and domain-specific intuition (how would they work for YOUR images), and hence we encourage the readers to experiment with the associated notebooks and use them for their own data.

Introduction to autoencoders

Prior to discussing the VAEs, we briefly introduce the principle of simple autoencoders (AEs) and their applications. Generally, AE represents a class

of neural networks comprised of an encoder and a decoder part. The encoder takes the input object, such as an image or image patch, and passes it through a set of convolutional or dense layers with progressively decreasing dimensionality down to a bottleneck or latent layer. In this process, the dimensionality of the data is reduced from the original size to a usually very small number of latent variables. The decoder then takes the latent vector and passes it through a set of layers of increasing dimensionality to reconstruct the original data. Often but not necessarily the architecture of the encoder and decoder are symmetric with respect to the bottleneck layer. The training process for autoencoders aims to minimize the difference between the input and reconstructed data, which helps the autoencoder learn to effectively compress and reconstruct the original dataset. The loss function in the AE can be represented in Eq. 1, where X is the input image and $decoder(encoder(X))$ is the reconstructed image and LF is the specific loss function chosen for a given dataset.

$$Loss = LF(X, decoder(encoder(X))) \quad (1)$$

In principle, AE can be trained with many loss functions, including but not limited to cross entropy loss, mean absolute error, and mean squared error. The structure of the encoder and decoder can follow any of the commonly used machine learning paradigms, such as fully connected multilayer perceptrons³⁴, convolutional networks³⁵, attention layers³⁶, and graph networks³⁷. The choice of architecture is determined by factors such as the type of input data (e.g., spectra, images, or graph representations) and the requirements of the specific domain. Similarly, the structure and architecture of the encoder and decoder can be different. However, the dimensionality of the input and output data are kept the same – alternatively, we have an encoder-decoder architecture rather than an autoencoder where the output is different from the input (e.g., images to spectra^{38–40}). These architectures also have multiple applications in establishing structure-property relationships, quantitative structure- activity relationships, etc. but are outside the scope of the present article.

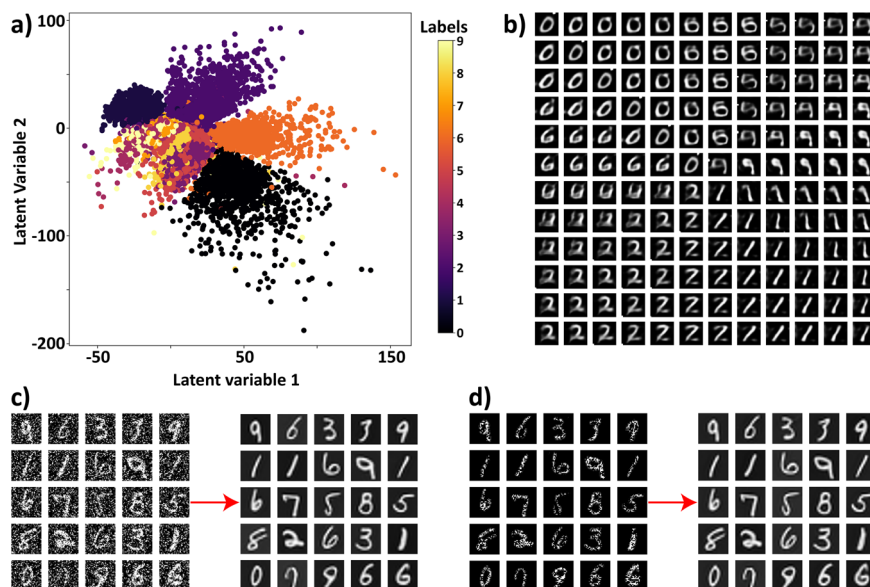
At first glance, autoencoders seem to be a very counterintuitive application—after all, they try to reconstruct the data itself. However, real power of the autoencoders is their capability to encode high dimensional data in the form of a low-dimensional vector and decode the data from the latent vector. In other words, it builds the relationship (input) $\rightarrow (l_1, \dots, l_n)$ and $(l_1, \dots, l_n) \rightarrow$ (output), where input and output are ideally as close as possible.

This transformation allows for several useful properties and applications. First, encoding and decoding the data via. latent vectors select the relevant features in the data and reject the spurious elements. As a toy dataset to demonstrate this, we have chosen the MNIST dataset⁴¹. MNIST dataset is a collection of images of handwritten digits between [0, 9] and is usually the go-to dataset for testing the computer vision algorithms. The results of application of AE on the MNIST dataset are shown in Fig. 1. The encodings of the inputs in a low dimensional latent space can be visualized in two different ways. Firstly, each input image can be plotted using a scatter-point in the low dimensional dataspace using their latent vector. In this method, all the input images are represented in a single scatter plot based on their corresponding location in the latent space. Additionally, we can color the points using their class label and see how different classes are clustered in the latent space. We will refer to such plots as the latent space distributions from hereon.

In the case of the MNIST dataset, latent space dimensionality is set to 2 and its corresponding latent space distribution is shown in Fig. 1a, where the scatter-points are colored based on their class [0–9]. The latent space distribution shows that, even without any prior information about the class labels, the images belonging to each class are clustered in the latent space. Furthermore, the classes that share some common features overlap with each other.

The second approach to visualize the construction of latent space is shown in Fig. 1b. In this approach, the input dataset is discarded after the training phase. The trained latent space is then uniformly sampled, and then

Fig. 1 | Overview of autoencoder applications on the MNIST dataset: Demonstrating dimensionality reduction, noise cleaning, and pixel completion. **a** MNIST dataset represented in the 2D latent space and colored using the class labels, **b** uniformly sampled and decoded latent space, **c** representation of denoising the MNIST dataset using AE, the noisy inputs to the AE and corresponding cleaned images are shown, **d** representation of filling the missing pixels application on the MNIST dataset using AE, the inputs with missing pixels and the reconstructed outputs from the AE are shown.



decoded back into the input space. These decoded images are then plotted in their corresponding positions in the latent space. This way of visualizing the latent space will be referred to as the decoded latent space, and is shown in Fig. 1b for the MNIST dataset. Decoded latent space helps us to visualize the characteristics of the latent space without using the input dataset.

For the MNIST dataset, the decoded latent space shows how the image classes are distributed in the latent space and the overlapping of features from different image classes. In AE, the latent space is not regularized, and the limits used in each of the latent dimensions to uniformly sample the latent space are obtained from the latent distributions. For the MNIST case, the limits for the decoded latent space are obtained for the latent distribution in Fig. 1a and are set to $[-100, 100]$ for both latent dimensions. The axis labels on the decoded latent space are omitted in certain places to make the figures legible. These decoded latent spaces show the variation of the inputs in the latent space. Additionally, since the latent space is uniformly sampled, the absolute values of the latent variables do not contribute to additional understandings.

AEs are extensively used for denoising and data reconstruction along with dimensionality reduction. For these and many other applications, the key element becomes the choice of inputs and outputs. For example, in case of denoising, a noisy dataset is used as the input and the loss function in Eq. 1 is evaluated between the reconstructed dataset (output) and a clean dataset. This method works if we have access to the noise-free (clean) dataset beforehand. One way to overcome this limitation is to create a synthetic dataset using domain expertise so that it closely represents the dataset at hand. One can then add noise to the dataset and train the autoencoder on the synthetic dataset. This trained network can then be used to clean the original dataset. We demonstrated this by applying noise to the MNIST dataset and used the AE architecture to clean the noisy images. The inputs (noisy) and outputs (clean) for this process are shown in Fig. 1c.

Another application of autoencoders is image reconstruction, i.e., inpainting missing pixels or restoring color. In the missing pixels case, input to the autoencoder is the dataset with incomplete data and output is the cleaned and complete dataset. The missing pixels inputs of the MNIST dataset, and their corresponding outputs are shown in Fig. 1d. The analyses represented in Fig. 1 are available with code in 'Intro to AE and VAE.ipynb' jupyter notebook that accompanies the manuscript. Each section has once such notebook in the publicly accessible GitHub repository, where the dataset used in the analysis can be downloaded into the notebook. The reader can reproduce the entire analysis corresponding to the results discussed in the manuscript. In cases where training the architecture from

scratch is either time or memory extensive, pre-trained weights are loaded in the jupyter notebook to accelerate the process.

It is also very important to note that this process implicitly illustrates the limitations of autoencoders—the missing information is not coming from nowhere—it in fact is derived from the correlations between the features in the original dataset. If the objects with the shape of the cars were red in the training dataset, then recolored cars will also be red in the reconstructed dataset. However, if the dataset was acquired during the Henry Ford times ('you can buy the car of any color, as long as it is black'), the reconstruction will be misleading. This is one of very significant limitations of the neural networks applied to reconstruction and super-resolution problems, these and other limitations will be repeatedly pointed throughout the text.

Introduction to VAEs

With the above-mentioned elucidation on AEs, we now discuss VAEs, the principal topic of this article. The encoder and the decoder part of VAE look similar to those of autoencoders, with a significant difference being the structure of the latent layer. In this case rather than rigidly pass the data through the latent bottleneck, the VAE samples the latent space. An excellent introduction into principles and properties of VAEs are available in the publication by Kingma³².

VAEs have the same properties as autoencoders AEs, but also have the ability to disentangle the latent representations of data. While the formal definition of this process is still debated, it is generally understood to involve the association of different factors of variation in data with different latent variables. Variational Autoencoders (VAEs) have been widely recognized for their generative capabilities, where they can generate unseen data samples that are similar in distribution to the training data, making them a powerful tool in the realm of unsupervised learning. By sampling a latent point that lies between the latent representations of two input data samples, VAEs can produce unseen data samples that have features from both input data points and are not part of the input dataset. These generated samples can be thought of as a smooth interpolation between the input data samples in the latent space, and they can provide valuable insights into the structure of the input data distribution.

The MNIST dataset used in elucidating AEs is also used here to showcase the capabilities and shortcomings of the traditional VAE construction. We use the decoded latent space and latent distributions as the visualization tools for analyzing the results of VAE. Decoded latent space and latent distribution of the VAE trained on the MNIST dataset are shown in Figs. 2a, e, respectively. The limits for plotting the decoded latent space of

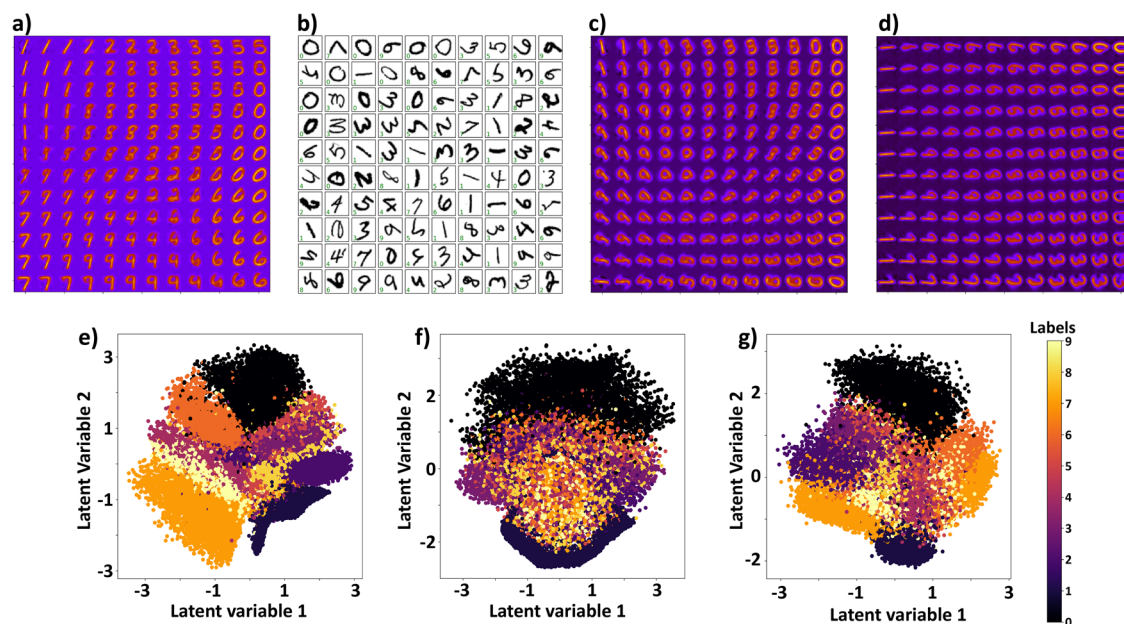


Fig. 2 | Comparing and contrasting the results of VAE and rVAE on regular MNIST and rotated MNIST datasets. **a** Decoded latent space of VAE trained on the regular MNIST dataset, the limits on each latent dimension are $[-1.5, 1.5]$ for all VAE decoded latent spaces unless otherwise specified, **b** Examples from the rotated MNIST dataset where the angle of rotation of each image is randomly sampled from the interval $[-90^\circ, 90^\circ]$, **c** Decoded latent space of the VAE when trained on the

rotated MNIST dataset, **d** Decoded latent space of rVAE when trained on the rotated MNIST dataset. Latent distributions colored using the class labels of the images for **e** VAE trained on MNIST dataset, **f** VAE trained on the rotated MNIST dataset, and **g** rVAE trained on the rotated MNIST dataset. The color bar for (e–g) is the same and is shown in (g).

the VAE and its variants for the rest of the article will be $[-1.5, 1.5]$ unless otherwise specified. These limits work because of the prior distribution enforced on the latent variables. The prior distribution is usually considered to be independent zero-centered normal distribution with unit variance which will be further discussed in the formalism section. The decoded latent space of the VAE shows how the important features of the dataset are encoded in the latent space. For example, all the straight lines segments involved in the dataset ('1's and '7's) are towards the left of the dataset which slowly transform to the circles towards the right of the dataset ('0's, '6's, and '8's). The latent distribution shown in Fig. 2e shows how the latent space is regularized, i.e., different data classes are tightly packed in the latent space, with very narrow boundaries of 'unphysical' structures between the regions corresponding to different digits. This paves way for the generative capabilities of VAE.

Shortcomings of the traditional construction of VAEs can be illustrated by adding additional variability (rotated MNIST) to the dataset. Generally, we choose this special case since in imaging (unlike the human handwriting) the orientation of the object in the image plane is apriori unknown. In the rotated MNIST dataset, each image is rotated by an angle that is randomly sampled from the range $[-90^\circ, 90^\circ]$. Randomly sampled images from the resulting dataset is shown in Fig. 2c which act as the input to a different VAE network. The decoded latent space and the latent distributions of the VAE trained on this rotated dataset are shown in Fig. 2c and f, respectively. The decoded latent space (Fig. 2c) encodes features at varying angles, as well as the specific features discussed in the unrotated dataset case. The latent distributions in Fig. 2f are colored by the class labels of the dataset. The class labels in the latent space are now ambiguous due to rotational symmetries present in some of the classes. The latent distribution shows that the VAE does a sub-optimal job in segregating the classes as no clear clusters for each class are visible. Increasing the number of latent variables in this analysis might help but there it is not guaranteed that the latent space disentangles the angle involved in producing the image.

To counter this problem, we used a rotationally invariant variational autoencoder (rVAE) to analyze the rotated MNIST dataset. This class of the variational autoencoders ensures that one of the latent variables is the angle

of rotation of the object in the image. This angle-invariance (as well as translation-invariance) will be discussed in more detail in the section on rVAE. The decoded latent space and the latent distributions of the rVAE are shown in Fig. 2d, g, respectively. The results of rVAE on the rotated MNIST dataset are shown here as we will use additional datasets to explain the workings of different classes of variational autoencoders after this section. The decoded latent space of the rVAE (Fig. 2d) shows that all the digits are encoded with respect to the horizontal axis in a single orientation. The horizontal orientation in the latent is because the zero of the encoded angles is arbitrary in the rVAE. The latent distributions (Fig. 2g) show clusters of different classes in the latent space distribution which shows that the rVAE does a better job at separating different classes in comparison to vanilla VAE. The analyses that constructed the Fig. 2 are available with code in 'Intro to AE and VAE.ipynb' jupyter notebook that accompanies the manuscript.

Mathematical formulations of VAE

Variational Autoencoder (VAE) is a generative model that learns the latent space of high-dimensional data in an unsupervised way. VAE consists of two parts: an encoder $q_\tau(z|x)$ also known as the inference network that is parameterized by τ , and a decoder $p_\phi(x|z)$ also known as the generator network parameterized by ϕ . The posterior distribution $p(z|x)$ is approximated by a tractable distribution which is often referred to as the variational distribution. For the analyses in this article, the variational distribution is a Gaussian distribution whose parameters are predicted by the encoder $q_\tau(z|x)$. The decoder when provided with a latent space vector (z), outputs the parameters to the likelihood distribution $p(x|z)$. The goal of the VAE is to learn a compact and smooth latent space that can be used for generating unseen data samples.

To achieve this, VAE introduces a prior distribution $p(z)$ over the latent space, typically a standard Gaussian distribution $\mathcal{N}(0, I)$ with zero mean and unit variance. This prior distribution acts as a regularizer, ensuring that the latent space is compact. The encoder network maps the input data to a mean and a standard deviation of a variational Gaussian distribution in the latent space.

To train a VAE, the Evidence Lower Bound (ELBO), a loss function in variational inference is maximized. The ELBO for VAE consists of two terms: the reconstruction loss and the KL divergence between the learned latent distribution and the prior distribution. The reconstruction loss measures the difference between the input data and the reconstructed output, while the KL divergence encourages the learned latent distribution to be close to the prior distribution. The VAE learns the optimal encoder and decoder networks by maximizing the ELBO given by Eq. 2.

$$ELBO_{vae} = -\text{Reconstruction error} - D_{KL}(q(z|x) \mathcal{N}(0, I)) \quad (2)$$

In summary, the VAE is a generative model that learns a smooth and continuous latent space by introducing a prior distribution over the latent space and maximizing the ELBO. The encoder maps the input data to a mean and a standard deviation of a Gaussian distribution in the latent space, and the decoder maps the latent space to the output data. The VAE is trained by minimizing the reconstruction loss and the KL divergence between the learned latent distribution and the prior distribution or by maximizing the ELBO.

VAE trained on the cards datasets

With the discussions of mathematical foundations, we now proceed to test the capabilities of VAEs on a toy dataset that will be used to evaluate different versions of VAEs from here on. The toy dataset we have chosen for this purpose is of playing cards, with monochrome images of all 4 suits viz., clubs, spades, diamonds, and hearts. Despite its simplicity, the cards dataset forms an interesting collection of symbols. For example, clubs (without tail) have three-fold and mirror symmetry. Moreover, horizontally flipped hearts and spades only differ by small details which makes it hard for the unsupervised algorithms to differentiate between them. Diamonds, on the other hand have a shape that is distinct from other three suits. Rotating the diamonds suit by 90° is equivalent to uniaxial compression and resizing. These phenomena introduce degeneracies in the outcomes of affine transformed cards dataset. With these profound complexities in terms of symmetries, the cards dataset poses to be a challenging one for the machine learning algorithms—but at the same time the relevant behaviors can be readily illustrated in the 2D latent spaces, allowing for intuitively clear explanations.

Variability in the images of the cards dataset is introduced using different affine transformations on each image. Each image is rotated using an angle that is randomly sampled from the range $[-12^\circ, 12^\circ]$ for the datasets with low rotations and $[-120^\circ, 120^\circ]$ for the datasets with high rotations. The images are additionally sheared equally in both x and y directions where the value of the shear is sampled from $[-1^\circ, 1^\circ]$ for the low shear case and $[-20^\circ, 20^\circ]$ for the high shear case. In total, four datasets are formed with combinations of these two transformations. The datasets thus formed are (i) low rotations and low shear, (ii) low rotations and high shear, (iii) high rotations and low shear, and (iv) high rotations and high shear. These datasets will be used to analyze different types of VAEs in the upcoming sections and will be referred to as cards dataset (i–iv) for brevity.

One VAE per dataset is trained and the corresponding results of all four datasets are shown in Fig. 3. The figure is divided into 4 parts viz., (p–s) corresponding to cards datasets (i–iv). Twenty-five randomly chosen examples from each dataset are shown in Fig. 3a. The decoded latent spaces of all four datasets are shown in Fig. 3b. While the latent distributions of all four datasets are shown in Fig. 3c where the scatter points are colored using the ground-truth class label of each datapoint. The limits for x and y axes in the latent distributions are omitted for brevity since the plots are only used for visualizing the clustering of suits in the latent space. It can be observed from the latent distributions that the VAE does an optimal job in encoding the images into the latent space with low rotations (Fig. 3cp and Fig. 3cq). Clusters of four different classes in the dataset and the boundaries between them are clearly visible. Similar conclusions can be drawn from the decoded latent spaces (Fig. 3bp and Fig. 3bq) of these two datasets. Additionally, since the latent space of VAEs is smooth in terms of physical variabilities, each

cluster is expected to have a smooth variation in angles and shear in the latent space. In the decoded latent spaces of these two datasets, each image unambiguously belongs to one of the four classes which also aids us in making the conclusion that the VAE can generate optimal encodings for all the classes in the latent space for the first two datasets.

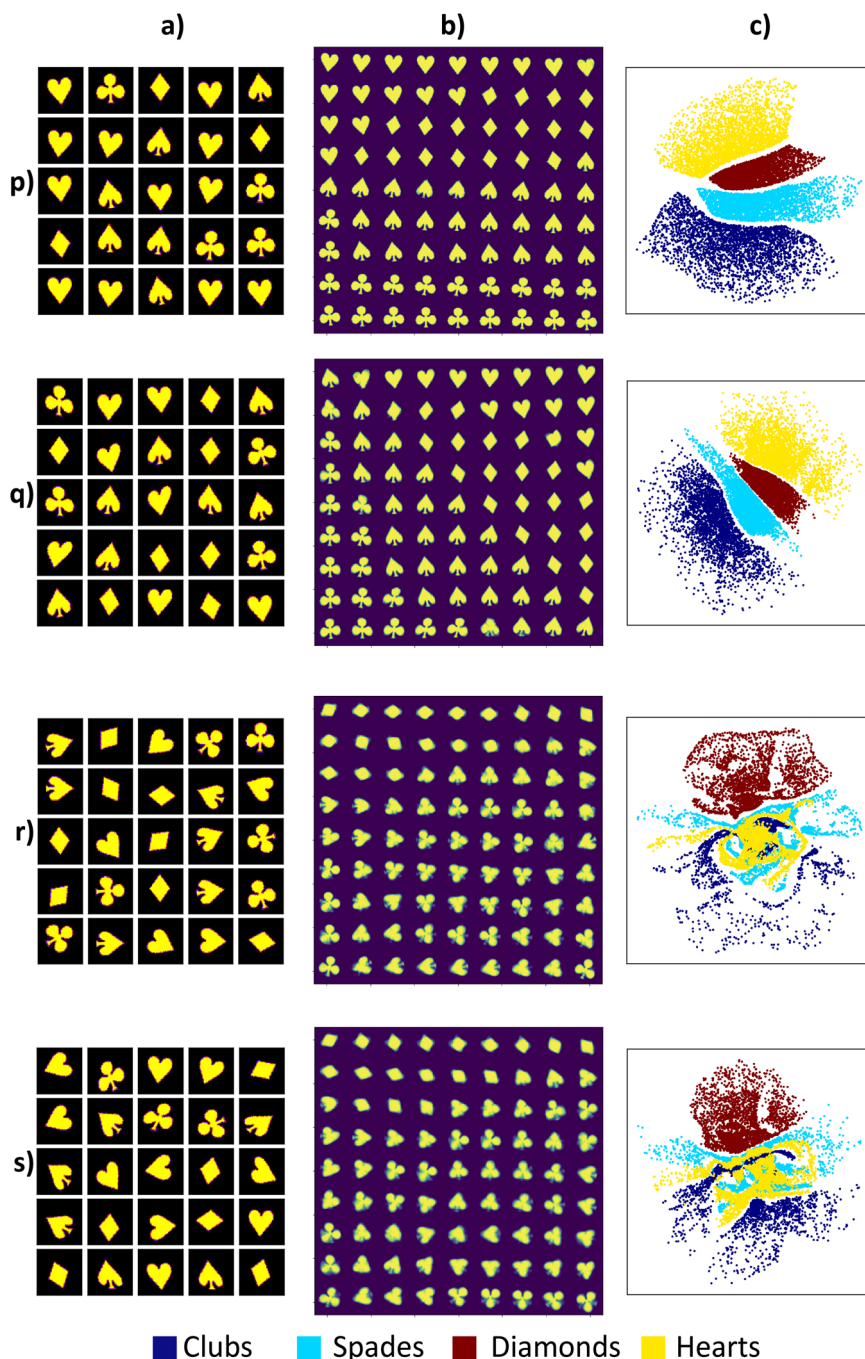
This is not the case for the latter two datasets (cards – iii and iv) and the results of these two datasets are shown in Figs. 3r and 3s respectively. The latent space distributions (Fig. 3cr and Figure 3cs) show that only the diamonds class form a clear cluster and there is a considerable overlap between the other three classes. This overlap arises from the degeneracies present in the input dataset at high rotations. The diamonds do not have a significant overlap with the other three classes in terms of their structure. This can also be observed in the decoded latent spaces (Fig. 3br and Figure 3bs) where we have images that have characteristics from two or more classes except for diamonds which occupy the top rows in both the cases. This analysis elucidates the short comings of VAE on the datasets with rotations especially in the presence of rotational symmetries. The complete analysis can be visualized and reproduced using the ‘intro to AE and VAE.ipynb’ jupyter notebook that accompanies this article.

Choosing the descriptors

The key element in the application of VAE for imaging data is the choice of descriptors. For spectral imaging data $R(x, y, t)$, where x and y are coordinates. The natural object is the spectrum $R(t)$ at location $[x, y]$, where the t is a descriptor that can be a physical parameter extracted from the spectra. For imaging data, there are multiple strategies for selecting the descriptors. One of them is to hand-pick the physical quantitative descriptors from the dataset based on the domain expertise. To elucidate this process, we consider the Piezoresponse Force Microscopy (PFM) image data showing ferroelectric domains of PbTiO_3 (PTO) at two different times t_0 and t_1 in Fig. 4a–d. The amplitude responses of the PFM for both times are shown in Fig. 4a, c respectively. The respective domain walls are semantically segmented using a deep convolutional neural network and are shown in Fig. 4b, d respectively. In this case, the physical descriptors can be (1) the polarization state in a region, e.g., the ratio of the bright region P_1 and the dark region P_2 in Fig. 4a, c; (2) number of domain walls, e.g., there are two domain wall W_1 and W_2 in the images Fig. 4b, d; (3) the distance of two nearest domain walls, e.g., d_1 and d_2 in Fig. 4b, d. A different way of selecting the quantitative information from the image dataset is to use the neighborhood of the atoms in the atomically resolved Scanning Transmission Electron Microscope (STEM) image. However, the efficiency of this method depends on presenting the entire information of the neighborhood in a format that can be used by the ML techniques. Molecular graphs⁴², SMILES⁴³, SELFIES⁴⁴ are some of the well-known techniques to encode the neighborhood information in a format that can be handled by ML/DL techniques. These methods of selecting hand-picked descriptors or encoding neighborhoods oftentimes fail to include the entire information present in the dataset and/or result in unintended biases.

The simpler way of selecting the descriptors of the system is the construction of the sub-images centered on the square grid within the original image. The density of the square grid determines the number of sub-images sampled from a given image. The schematic of this method is shown in Fig. 4f. This method precludes any user-specific biases during the construction of the descriptors. The construction of these sub-images is straightforward. This approach has been used in several previous studies^{45–47}. However, for images containing object of interest the relationship between the patch center and the object can be arbitrary. Correspondingly, these random shifts become the primary factors of variation within the data. For example, Fig. 4a–d shows a PFM data of ferroelectric PTO film, during the PFM measurement, a DC bias is applied through the tip to switch the polarization. The polarization state variation and domain wall motion can be recorded by continuous PFM image scan. Figure 4a shows a frame at time t_0 of this continuous PFM dataset. Sub-images can be extracted from this dataset, an example corresponding the red square marked region is shown in Fig. 4a. Since the polarization state and domain wall are changing under the

Fig. 3 | Results of the vanilla VAE on four cards datasets formed with varying limits on the rotations and shears applied to the card images. The rows in the figure i.e., (p-s) correspond to the different cards datasets (i–iv) formed with the application of transformations on the images of the cards respectively. **a** Twenty-five randomly chosen images from each dataset to visualize the intricacies in each dataset, **b** decoded latent spaces of each dataset, and **c** latent distributions in the latent space for each dataset where the scatter points are colored using the original class labels.



application of DC bias, after a certain time (e.g., at time t_1), a polarization state change (Fig. 4c) and a domain wall shift (Fig. 4d) can be observed. This can be partially offset by the shift-invariant VAE⁴⁸ approach as will be discussed below. However, shift-VAE is limited when the object shape changes due to overlap with the patch edge, etc.

The second approach for construction of sub-images combines the patch generation with the physics-based descriptions. For STEM data, it is natural to define patches centered at the individual atoms (Fig. 4g). In this case, the patch collection contains the full information in the image, but via., construction, analyzes the neighborhoods of individual atoms. This in turn necessitates introduction of rotationally invariant autoencoders, since otherwise the objects can have arbitrary orientation in the image plane.

Finally, for the time-dependent dynamic data the descriptors can be constructed to contain time information. For example, when analyzing the ferroelectric domain wall motion, the patches can be centered at the time-

lapsed images. In this manner, the descriptors will contain the information on the time dynamics^{49,50}.

Building invariant VAEs

Examples of the VAEs discussed in the previous sections illustrated the principles of VAE and the choice of descriptors helps in elucidating the phenomena of interest. However, applications of VAEs for image analysis is associated with additional challenges. As discussed above, ultimately, we aim to discover the factors of variability in data or disentangle the data representations. However, in real images multiple factors of variability will be associated with the positions of the object of interest, rotations, or small changes of scale. For example, when identifying nanoparticle shapes from microscopy images, we generally assume that nanoparticle will be the same down to rotation in image plane. At the same time, uncertainties in establishing anchor points when choosing descriptors will result in the presence

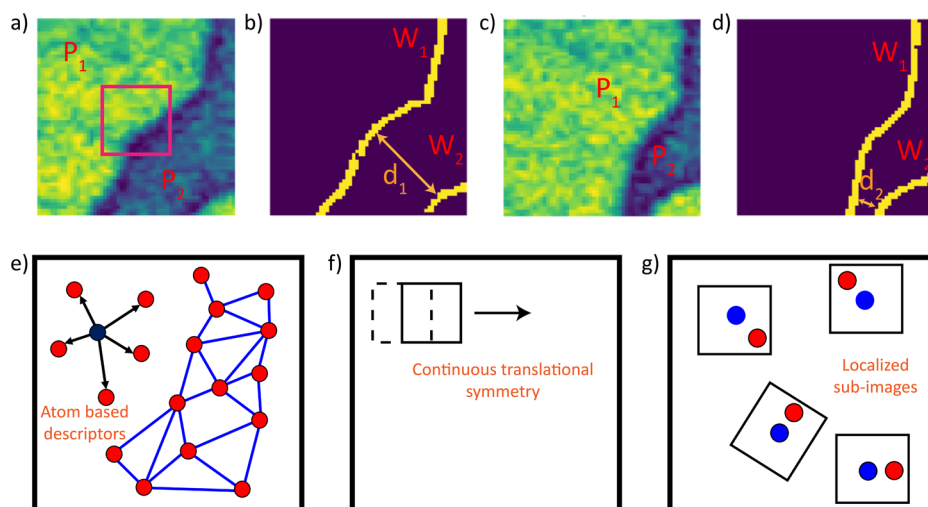


Fig. 4 | Demonstration of multiple strategies for choosing the descriptors from image data. **a–d** Piezoresponse Force Microscopy image of ferroelectric PTO film, adapted from ref. 49 PFM amplitude image at time **a** t_0 and **c** t_1 , domain wall images generated by DCNN corresponding to time **b** t_0 and **d** t_1 . Schematic of choosing the neighborhoods of atoms in atomically-resolved images as descriptors. **e** A graph of where the nodes are atoms in the image and the nearby atoms are connected by an edge can act as an input to graph neural networks. Or the feature engineering in the

VAE image analysis can be based on the selection of image patches. These can be based on rectangular sampling grid (**f**) (sliding window approach) or be centered on individual features such as atoms (**g**) or domain walls (red square in (**a**)). For dynamic data, the feature can be centered on the time-delayed image. Finally, the patches can be analyzed with or without rotational, translational, or other symmetries depending on the physics of the particular problem and imaging process.

of translational offsets in images. These considerations prompt the development of invariant VAEs, generally referring to the network architectures that separate physical factors of variability (translation, rotation, scale) in separate latent variables, and disentangling the factors of variability in remaining degrees of freedom.

Introducing invariance

In this section, we explore the construction and workings of rotationally invariant autoencoders, which we address as rVAEs from hereon for brevity. In the traditional setting of VAE, the latent variables encompass the essential features of the input dataset and are usually not controlled by the user. For example, in the MNIST dataset, the input datapoint's writing style and class (0–9) are the dominant features learned by the latent space. The rotations and translations of the object present in the images are treated as the dataset's features. The latent space learns them by treating them as input variations along with other features. For example, a single digit from the MNIST dataset can be rotated and translated randomly. And when a regular VAE is trained on this dataset, the angles and the translations are expected to be encoded in the smoothly varying latent space. In this process, the VAE treats the same object with different rotational and translational transformations as separate entities and produces different encodings for them. The rVAE network is designed such that 1–3 latent dimensions are used to explicitly encode the rotations and/or translations present in the input image. By incorporating both rotational and translational invariance in the rVAE network, the network can explicitly encode both rotational and translational information in the latent space, providing a more robust and accurate representation of the input data. For example, the encodings of images with similar objects appearing at different orientations will be similar in the latent space except for the encoding in the dimension that explicitly encodes the angles.

Rotations and translations are two important aspects of the images we encounter in our daily lives, as well as in many scientific applications including materials science and condensed matter physics. The ability of the rVAE network to capture both rotational and translational information can be particularly useful for analyzing microscopic images, which helps researchers to better understand the underlying rotational and translational patterns and their impact on material properties. Similarly, in condensed matter physics, rotations and translations play a crucial role in

understanding the behavior of particles at the atomic and molecular scale. Overall, the incorporation of both rotational and translational invariance in the rVAE network provide a powerful tool for analyzing and generating diverse images in various scientific applications, enabling researchers to better understand both the rotational and translational patterns and dynamics in their data.

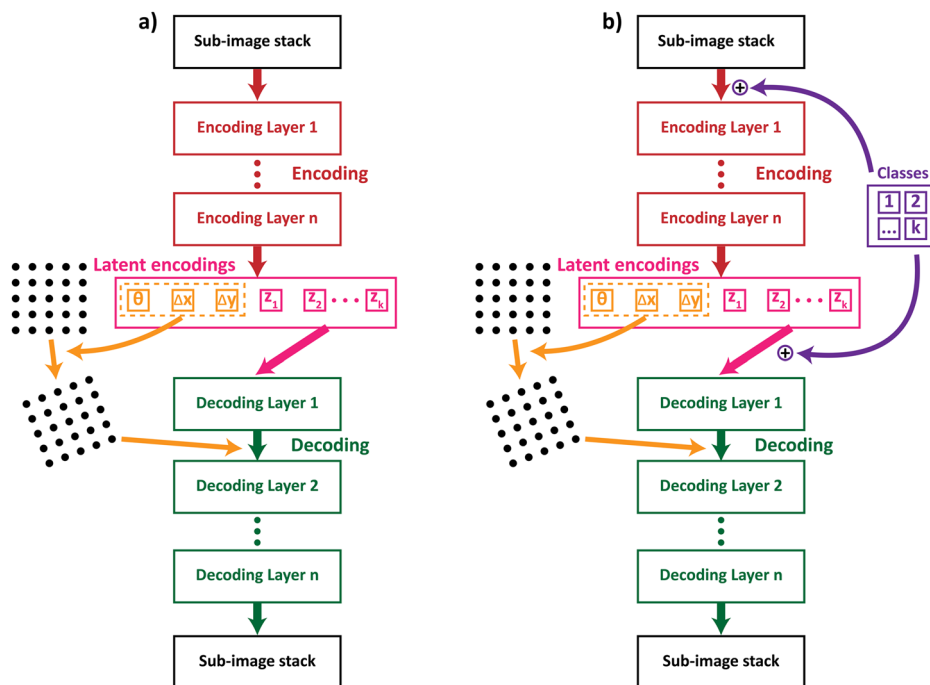
The encoder part of rVAE is similar to the traditional VAE, where a fully connected dense neural network (DNN) or a convolutional neural network (CNN) progressively decreases the dimensionality of the dataset until we reach a bottleneck layer of k -dimensions. We will refer to these latent dimensions of different variants of VAEs including rVAEs as $[z_1, z_2, \dots, z_k]$. These latent dimensions capture the variabilities in the input dataset and smoothly encode them in the latent space. The rVAE has the 1–3 additional features concatenated to this bottleneck layer corresponding to the angle (θ), x-translation (Δx), and y-translation (Δy), depending on the type of rVAE used for training. Hence the dimensionality of the latent space of the rVAE is $k + 3$ for implementing rotational and translational invariances, where k is the number of latent variables chosen by the user. rVAEs use a special decoder known as the spatial generator net introduced by ref. 51 in 2019. The spatial generator decoder net uses the coordinates of the input image along with the low dimensional latent representation of the encoder to decode the images. Initially, we form a 2D array of the coordinates of the pixels involved in the input image. The latent features corresponding to the rotational and translational invariances ($\theta, \Delta x, \Delta y$) are then used to rotate and translate the 2D coordinate array, and this transformation is given by Eq. 3,

$$x_t = x * \begin{bmatrix} \cos \theta & \sin \theta \\ -\sin \theta & \cos \theta \end{bmatrix} + [\Delta x, \Delta y] \quad (3)$$

Where x and x_t are the original and transformed coordinates, respectively, $\theta, \Delta x, \Delta y$ are the rotational and translational encodings of the rVAE.

The first layer of the decoder projects the latent space encoding $[z_1, z_2, z_3, \dots, z_k]$ into a higher dimensional space. The transformed coordinates are then projected into this higher dimensional space using a different neural network. After the first layer, the higher dimensional z -vector is made into N copies, where N is the number of coordinates or pixels in the input image, and each higher dimensional coordinate vector is added to the copies of the

Fig. 5 | Working schematics of invariant variational autoencoders. a Rotationally invariant variational autoencoder (rVAE) and **b** conditional-rotationally invariant variational autoencoder (crVAE).



z -vector. After this operation, we have a higher dimensional encoding for each of the pixels. We then decode them using a fully connected DNN or a CNN like in the case of VAE. By making N copies of the z -vector, adding the higher dimensional coordinates to each of the copies, and decoding them back into the real space, the decodings can be viewed as conditional probabilities on each pixel where each decoder neuron is conditioned on the transformed coordinate. The schematic of rVAE is elucidated in the Fig. 5a. The parameters for the encoder and the decoder for rVAE are learned by maximizing the ELBO given by Eq. 4.

$$ELBO_{vae} = -\text{Reconstruction error} - D_{KL}\left(q(z|x) \mathcal{N}(0, I)\right) - D_{KL}\left(q(\theta|x) \mathcal{N}(0, \sigma_\theta^2)\right) - D_{KL}\left(q(\Delta x|x) \mathcal{N}(0, \sigma_s^2)\right) \quad (4)$$

Where reconstruction error is the error between the reconstructed image and the original image which can be computed using the loss functions like mean squared error, cross entropy, and so on; the second term refers to the KL divergence between the posterior and the prior ($\mathcal{N}(0, 1)$) of the latent representations; the third term refers to the KL divergence between the posterior distribution of the angle and the Gaussian prior on θ with zero mean and standard deviation σ_θ ; and the last term in the KL divergence between the posterior of translations and its normally distributed prior with a standard deviation σ_s . The parameters involved in the prior distributions of these latent variables are the hyperparameters of the network. The weights of the entire architecture are learned by performing gradient ascent on the $ELBO_{vae}$, as discussed in the VAE section.

rVAE trained on the cards datasets

To explore the capabilities of rVAE, we use the cards datasets i-iv discussed earlier in the VAE section. Datasets are constructed from the original four cards belonging to the four suits, namely spades, diamonds, clubs, and hearts, by randomly rotating and shearing them. The random values for rotations and shears are picked from the interval $[-\alpha, \alpha]$ and $[-s, s]$, respectively. The specific values for α and s for all four datasets are discussed in the VAE section. One rotationally invariant network per dataset is trained with 2 hidden layers in both the encoder and the decoder. The low shear cases i.e., cards datasets i and iii use 256 neurons in the hidden layers and the high shear cases (cards datasets ii and iv) use 512 neurons in their hidden

layers. The encoder-decoder architecture is symmetric throughout this article. The rVAEs are expected to explicitly disentangle the rotational features and form a smoothly varying latent space that encodes the other variations in the dataset, which in our case are the class labels and the angles used to shear the images. The first two cases with low rotations are not discussed in the manuscript as they are straightforward and can be found in the jupyter notebook that accompanies the manuscript. Additionally, a regular VAE did an adequate job for these two datasets where the latent space forms a smooth variation of the variabilities, and the suits were all assigned to different areas of the latent space. The results for the last two cases with high rotations are discussed in Fig. 6.

Figure 6a, b shows the example images from both the high rotations datasets with low shear (cards dataset iii) and high shear (cards dataset iv) applied to the input images. The decoded latent space for two different rVAEs trained is shown in Fig. 6c, d, respectively. For the decoded latent spaces corresponding to the invariant VAE, the latent variables that capture the angles and translations are set to zero. This gives us the insight into how the images from each class are encoded in the latent space without the rotational and translational transformations. The baseline zero of the decoded angles is a function of the class label of the input image. This can also be observed from the decoded latent spaces where each class has a different orientation in the latent space. The predicted angle of rotation vs. the actual angle involved in producing the input dataset are plotted in Fig. 6c, d for the datasets iii and iv respectively. The color of the points corresponds to the class label of each input image. Finally, the latent distributions for the 2 cases are shown in Fig. 6g-j. Here the latent distributions are colored using the original class labels in Fig. 6g, h, using shear value in Fig. 6i, j for the two datasets. The latent distributions show that the rVAE is able to distinguish the four different classes present in the low shear dataset (cards dataset iii) and formed clusters in the latent space.

The ambiguous images in the decoded latent space emerge because the sampled points reside midway between the points in the input dataset. The datapoints from different classes are not overlapped in the latent distributions plot. Only the hearts suit is identified as two different clusters. The reason for this can be seen in the decoded latent space (Fig. 6c) towards the left ($z_1 = -1.5$), where the hearts suit is decoded in two different rotations. One of these decoded hearts suits has similarities with clubs towards the top, and the other has similarities with the spades and can be found in the middle

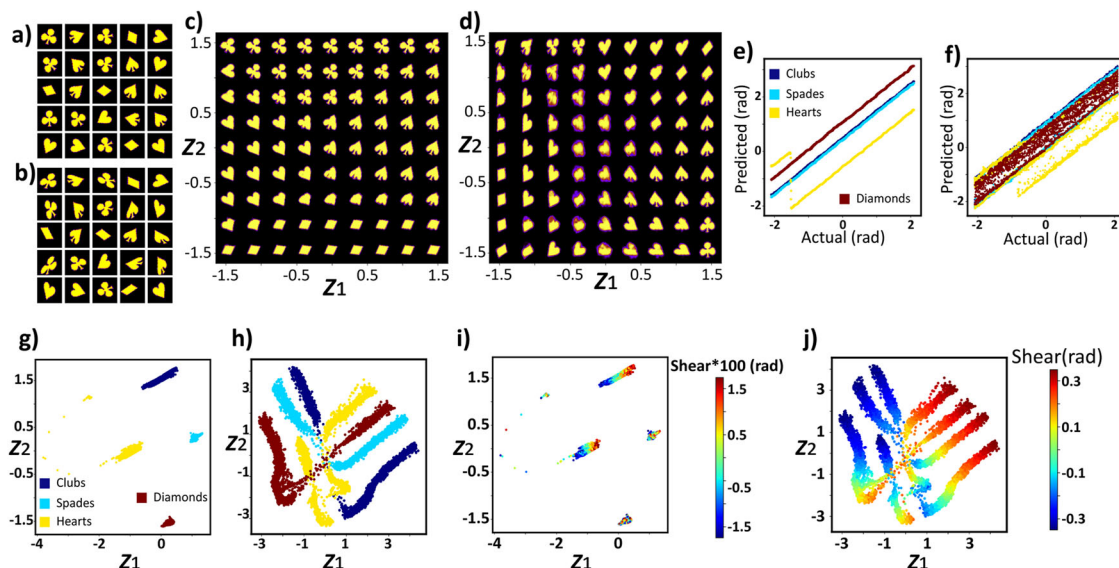


Fig. 6 | Results of the application of rVAE on the cards datasets. Twenty-five randomly sampled images form the **a** cards dataset iii and **b** cards dataset iv, decoded latent space correspond to the **c** cards dataset iii and **d** cards dataset iv, encoded angle vs. ground truth angle of the **e** cards dataset iii and **f** cards dataset iv where the points

are colored using the class labels, latent distributions of the **g** cards dataset iii and **h** cards dataset iv where the points are colored using the class labels, variation of ground truth shear value in the latent space for all the four classes corresponding to **i** cards dataset iii and **j** cards dataset iv.

of the decoded latent space. This can also be inferred from the angles plot (Fig. 6e) where two lines with different slopes are allocated to the hearts suit. It can also be seen from this plot that the baseline zero of the encoded angle is different for different classes. Finally, Fig. 6i shows that, for each class, the ground truth shear angle varies smoothly in the latent space. The deviation from this behavior is observed in cards dataset iii. This might be because the range of the applied shear is small compared to the angle of rotation for this dataset.

For the high rotation and the high shear dataset, the rVAE still outperforms the vanilla VAE by forming clusters in the latent space (Fig. 6h). Although the clusters formed in the latent space have overlaps between different classes, these overlaps are considerably smaller than the vanilla VAE case. The hearts suit overlaps with all the other three suits because of the similarities shared with the other suits. Towards the center of the decoded latent space all four suits overlap which can also be observed from the corresponding latent space (Fig. 6d), where the center of the latent space does not correspond to any one suit. Note that the manifolds corresponding to different classes cannot overlap (if objects are different), resulting in the characteristic ‘crossing’ pattern of different suits.

At such high rotations and shears, the structural difference between classes is substantially small. The decoded latent space (Fig. 6d) also shows a significant overlap between all four suits in this dataset leading to multiple ambiguous images in this plot. It also shows how image from one class smoothly emerge into an image from a different class. The decoded angle vs. the ground truth angle (Fig. 6f) shows the presence of ambiguity in predicting the angle. The inherent symmetries in the datasets along with high shear makes images indecipherable. The ground truth shear plot shows a smooth variation of the image attribute for every class, even with high shear transformation. Even though the encoding of this dataset is superior to that of vanilla VAE, the classification of suits is still ambiguous in the latent space due to the overlapping. This is because the applied transformations completely deformed the images, and even the rVAE cannot classify the input images using only the unlabeled data. The complete analysis and plots of the results for all four cases can be found in ‘rvaе_cards.ipynb’ jupyter notebook in the repository, accompanied by the manuscript.

rVAE trained on the experimental dataset

To understand the applications rVAE on an experimental dataset, we have chosen the STEM dataset of silicon in graphene manipulated by e-beam

activation^{52–56}. The dataset consists of 50 snapshots of the graphene system during the e-beam manipulation. The graphene system has both crystalline and amorphous regions along with several point defects around the individual atoms. Figure 7a shows the both the carbon atoms and silicon atoms (bright dots) in zeroth snapshot of the dataset. As an initial step to obtain feature vectors for the rVAE analysis, we pass the entire dataset through a pre-trained deep convolutional neural network (DCNN) to obtain the atomic coordinates. The details and the workings of this DCNN are thoroughly discussed in these refs.^{49,57} The DCNN is a semantic segmentation network that classifies every pixel in the images as belonging to one of the three classes viz., carbon atom, silicon atom, and the background. The output of the DCNN of the image shown in Fig. 7b where the three classes are shown in three different colors. The atomic coordinates are then obtained by calculating the center of masses of the pixels of each atom. These atomic coordinates are shown in Fig. 7c, where the carbon atoms are plotted as blue dots and the silicon atoms are plotted as bright red dots. These atomic coordinates are plotted on top of the original STEM image and as discussed earlier, the dataset of feature vectors for the identified atoms are constructed by cropping the image of a certain window size around these atoms. The window size is chosen such that it fits three hexagonal graphene rings around the central atom. It should be noted that the analysis that follows is sensitive to the window size used to create these feature vectors. The feature vectors are formed for all the identified atoms in all the snapshots which act as the input to the rVAE network. The rVAE network has 512 neurons in each of two hidden layers for both the encoder and decoder networks. We have used rotational and translational invariant version of VAE with two user selected latent dimensions (z_1 and z_2). This version of rVAE has a total of 5 latent dimensions viz., one dimension for encoding angle (θ), two dimensions for encoding translations in x and y directions (Δx and Δy), and two user selected latent dimensions (z_1 and z_2) to encode the neighborhood of each atom in the dataset.

After the training phase of rVAE network, the zeroth frame is re-plotted in Fig. 7d–f, where the atomic coordinates are now colored using the encodings of the feature vectors. Encoded angle, the first latent variable (z_1) and the second latent variable (z_2) are used for plotting the colors of the atomic coordinates in Fig. 7d, e, f, respectively. For an ideal graphene layer, there are two variants of atomic organization around the central atom rotated by 60° with respect to each other. The rVAE is able to identify the optimal rotational encodings for these two structures, as is represented by

Fig. 7 | Results of the application of rVAE on the graphene dataset. **a** Raw STEM image of the zeroth frame of the graphene dataset, **b** Semantically segmented output of the DCNN clearly discriminating two atom classes from the background class, **c** Atomic coordinates obtained from the center of masses of atom pixel clusters of the DCNN output. Each atom in the zeroth frame is colored using the encodings of **d** angle encoding (θ), **e** first latent variable (z_1), and **f** second latent variables (z_2) of the rVAE network.

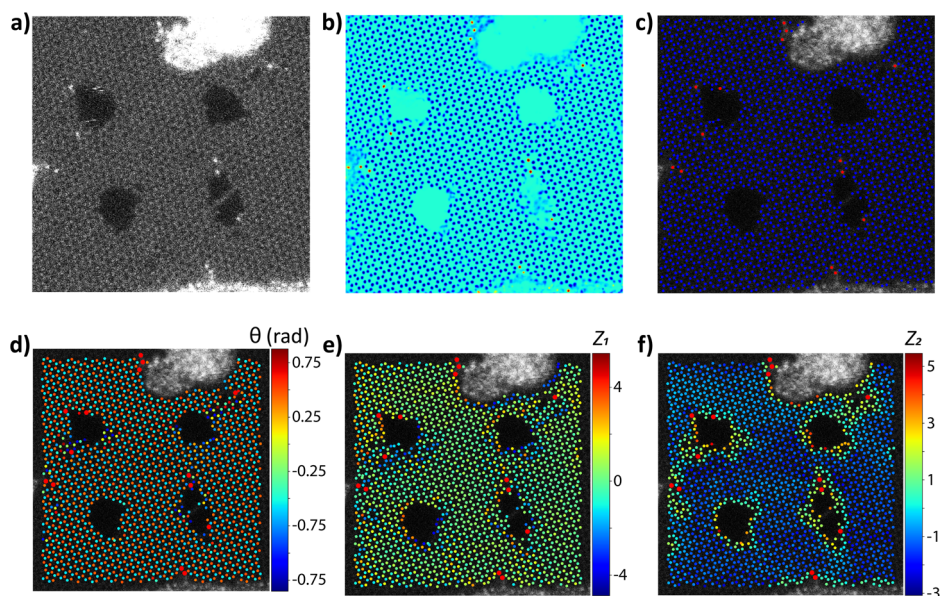
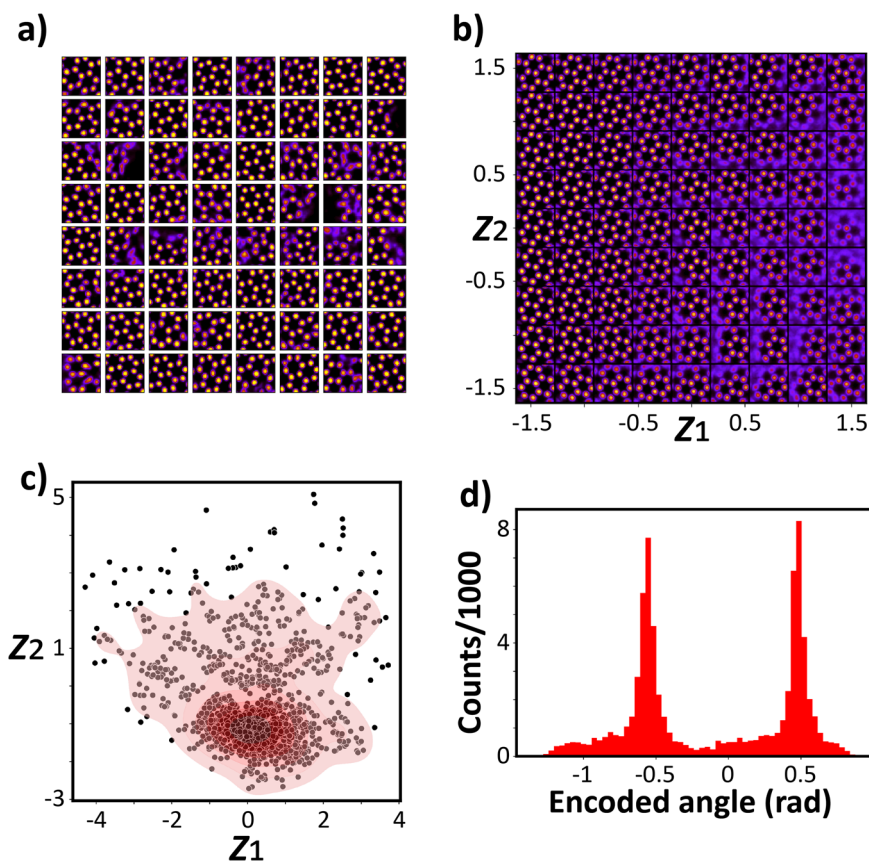


Fig. 8 | Decoded latent space and latent distributions of rVAE trained on the graphene dataset.

a Sixty-four randomly picked images from the input to the rVAE network, **b** decoded latent space of the rVAE network trained on graphene data, **c** kernel density plot of the latent distributions, **d** histogram of angular encodings (θ) in radians divided into 60 bins.



the checkerboard pattern in Fig. 7d corresponding to the two rotational variants. The second latent variable (z_2) in in Fig. 7f encodes the closeness to the amorphous region. It has a different value when the amorphous region is present in the cropped image of the atom which can be inferred by the sharp change in the color of this latent variable near the amorphous regions. First latent variable (z_1) seems to encode other forms of variability in the images like defects and strain in the feature vectors.

Further details of the rVAE encodings are discussed in Fig. 8. Sixty-four randomly picked images (feature vectors) from the input dataset are

shown in Fig. 8a, which gives us insights about the training dataset. The decoded latent space for this network is shown in Fig. 8b which elucidates the encodings as a function of the latent space. It can be seen from the decoded latent space that how certain areas encode the ideal graphene structures and the smooth variation into the images with amorphous regions. Although invisible to the naked eye, the other factor of variation in the images i.e., strain is also expected to be encoded smoothly in the latent space. Further analyses are required to show this which are beyond the scope of this article.

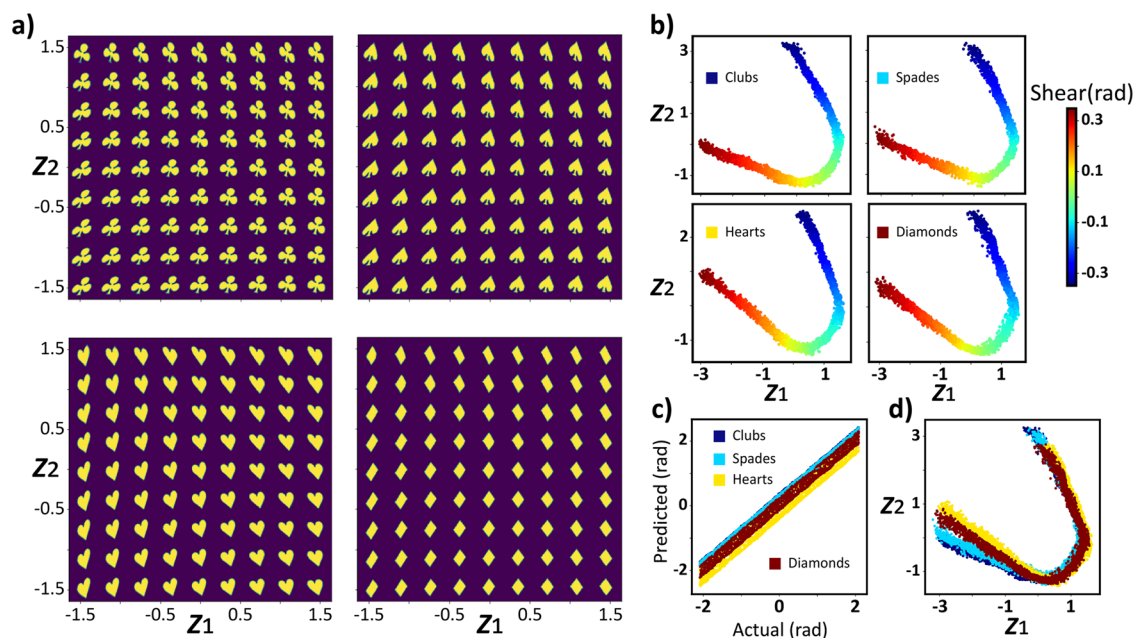


Fig. 9 | Results of the application of crVAE on the cards datasets. **a** Four distinct latent spaces formed as a result of four classes in the input cards dataset-iv, **b** latent distributions of four classes colored with ground truth shear value, **c** encoded angle

vs. ground truth angle used to rotate the input image, **d** latent distributions of four latent spaces shown in a single plot and colored using the ground truth labels of each datapoint. The colorbar for (d) is same as the one shown in (c).

Figure 8c shows the kernel density plot of the latent distribution (z_1 - z_2) of the atomic coordinates in the zeroth frame. The large cluster of points towards the center of the image corresponds to the ideal graphene structures as a majority of the input feature vectors correspond to them. The points away from the cluster are the feature vectors with defects and/or close to the amorphous regions. Finally, encoded angles for all the atoms found in the fifty frames are plotted as a histogram with 60 bins in Fig. 8d. The presence of two sharp peaks in the plot confirms that the two rotationally symmetric variants in the graphene layer are indeed found by the rVAE. These encodings in the 5 latent dimensions act as adequate feature vectors that encode both the angular component and the neighborhood independently for each atom. The discussions on the latent dimensions corresponding to the translational components is omitted for brevity. The entire analysis can be visualized and reproduced using the jupyter notebook 'rVAE_graphene.ipynb' that accompanies the article.

Conditional VAEs

In this section, we discuss conditional rotationally invariant variational autoencoder, which we refer to as crVAE for the rest of the manuscript. In the current construction of conditional VAEs discussed in this article, we assume that the input dataset is divided into discrete classes, and the class labels for each datapoint are known a-priori. Alternatively, condition can be continuous vector function. Generally, condition represents known factor of variability in the data. This helps the VAE to encode additional features given the information that we already know about the dataset.

The decoder network, also known as the generator network, learns to decode the encoded latent space conditioned on the class label of the data point. In the generator network, the encoded z-vector by rVAE is concatenated with the one hot encoded class label for the datapoint before decoding it with the spatial decoder network to form the crVAE. To make the encoder-decoder network symmetrical, we also concatenate the class label to the flattened input image. This has additional implications that are beyond the scope of this article. Since the decoder is conditioned on the class labels, one latent space per class is learned by the crVAE. It should also be noted that by the construction of rVAEs, the reconstructed output is independently conditioned on the rotation and translation encodings. Hence, for crVAE, the class labels can be sampled corresponding to one of

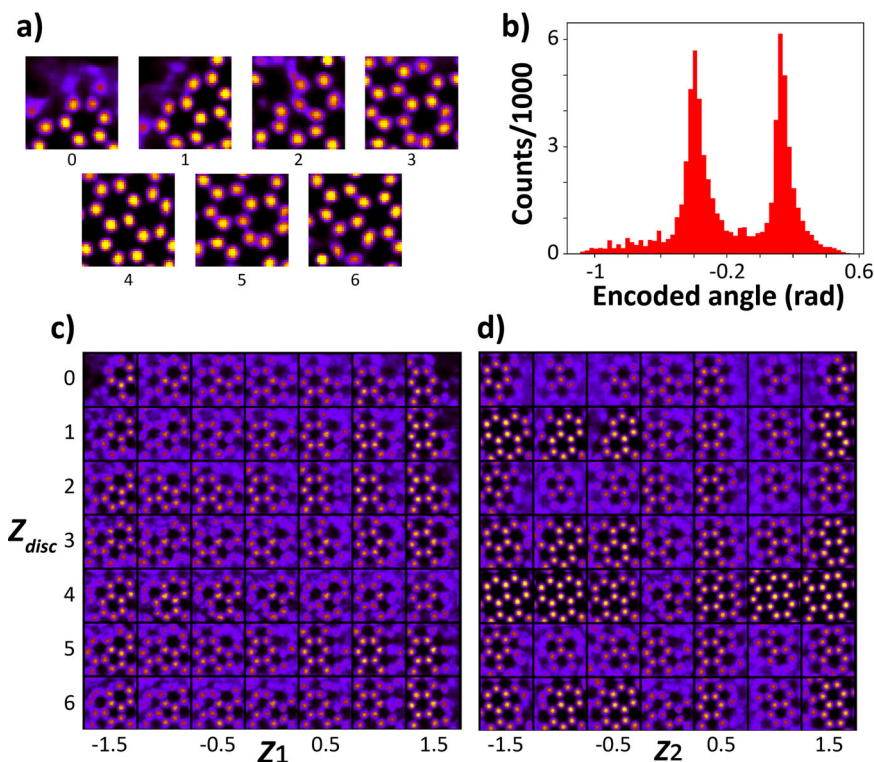
the latent spaces, then a latent vector corresponding to the rotational and translational encodings can be sampled along with a sampled point in the latent space to generate images using the decoder net. It is a salient feature of the construction of crVAE that the latent representations can be conditioned on either discrete class labels or the continuous ground truth values that produced the dataset (shear in the case of cards dataset). The ELBO loss function used for optimizing the crVAE is the similar to the one of rVAE and is shown in the Eq. 4 and the schematic of the workings of crVAE are shown in Fig. 5b.

crVAE trained on the cards datasets

To explore the machinery of the crVAE, we used the same four cards datasets discussed in the VAE section. For the architecture, we used 256 neurons in the two hidden layers of the encoder and the decoder for all four cases. The results for the first three cases are not discussed here as the explanations are straightforward and follow a similar trend to those of the fourth dataset. We chose the cards dataset iv as the rVAE in the previous section produced suboptimal encodings.

The crVAE analysis on the cards dataset iv are discussed in Fig. 9. The decoded latent spaces for each class of clubs, spades, hearts, and diamonds are in Fig. 9a. Since the class label is now not a feature of variability and the rotational component is already encoded by the rotational latent dimension θ , the only factor of variability left in the dataset is the shear value applied on the image. This shear value is expected to vary smoothly in the 2-dimensional latent space. This can be observed in latent distributions colored by the ground truth shear value in Fig. 9b for all the four classes. The encoded angle plotted against the ground truth angle and is shown in Fig. 9c. The encoded angle seems to be a single linear function of the ground truth angle with some variance which is due to the inherent symmetries present in the dataset. Finally, Fig. 9d shows the latent distributions colored with class labels. Although these the encodings of different classes seem to overlap with each other, the encodings of each class live in a separate latent space conditioned on the class label. Consequently, the crVAE emerges as a superior technique for encoding a dataset's inherent variabilities when specific information about the dataset is already known. This advantage is predicated on the prior knowledge of the class labels, which may not always be available. The entire analysis of crVAE on all four cards datasets (i-iv) can be

Fig. 10 | Results of the application of crVAE on the graphene dataset. **a** Representative datapoints for all seven classes used in the crVAE network for graphene dataset, **b** histograms of angular encodings (θ) divided into sixty bins, **c**, **d** traverse manifolds where one latent dimension along the x -axis and the class encodings along the y -axis are varied while the other latent variables are held constant at zero. First latent variable (z_1) is varied in (**c**), and the second latent variable (z_2) is varied in (**d**).



accessed through ‘crvae_cards.ipnyb’ jupyter notebook that accompanies the manuscript.

crVAE trained on the experimental dataset

To elucidate the workings of crVAE on an experimental dataset, we used the graphene dataset discussed in the rVAE section. Since crVAE prerequisites knowing a feature of variability of the dataset apriori, we divided the dataset into classes. The cropped images are placed into different classes based on the number and type of rings around the central atom. Every carbon atom in a graphene layer is a part of three hexagonal structures and only those three rings involving the central atom are used for classification. The manual classification task for this large experimental dataset is time consuming and hence an automated workflow to classify individual sub-images is proposed. Initially, a graph of atoms identified by the DCNN network is formed for each snapshot. An edge is formed between two atoms if the distance between them is less than a pre-defined threshold value. This threshold value is set so that only the three nearest neighbors of each atom are connected. To accommodate for the defects in the graphene layer, this distance is selected to be more than the ideal C-C bond with a restriction that each atom can have a maximum of three edges. The graph for each snapshot is formed in one go and 5-7 membered rings are then identified in the whole image by finding cycles for a given graph. A dummy channel is created in which the central pixel of each identified ring is populated with a value equal to the ring size. This dummy channel is then added to the original STEM image before cropping the sub-images to form the input dataset. Each cropped sub-image is then classified using the information from the dummy channel. Although one can form a large number of classes based on the neighborhood of the central atom only six classes with the highest number of datapoints associated to them are considered for analysis. It should be noted that although this method of classifying the images is less time-consuming it can be error-prone while forming the edges in the graphs or due to improper identification of the atoms in the STEM image.

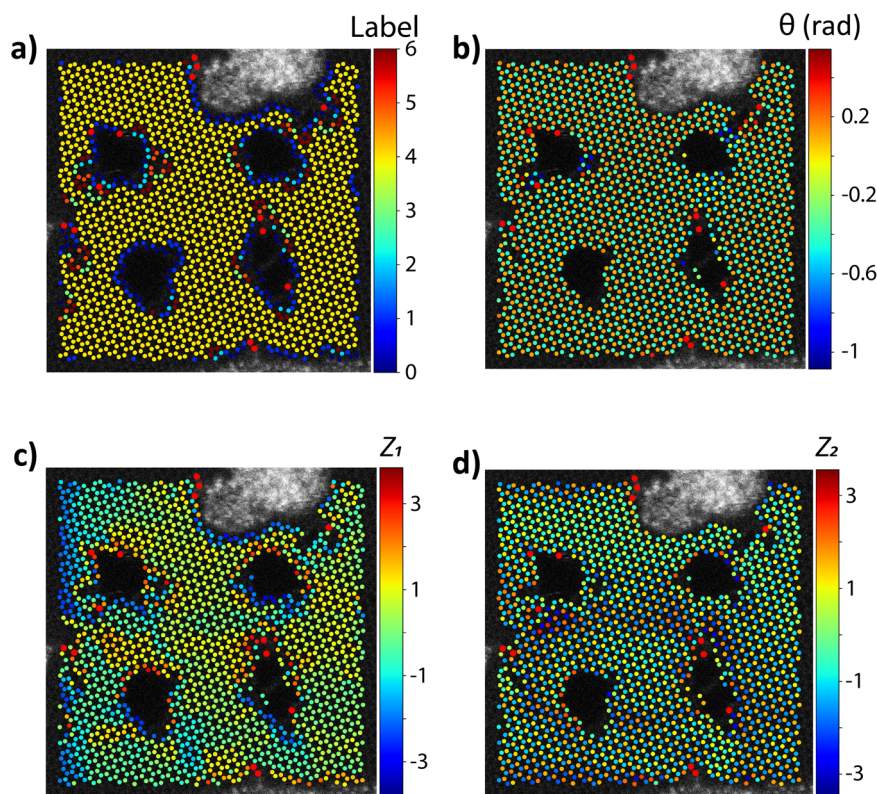
The seven classes considered for analysis are delineated in Fig. 10a which are 0: images with fewer than 2 rings, 1: images with 2 [6, 6] rings, 2: images with 2 [5, 6] rings, 3: images with 3 [6, 6, 7] rings, 4: images with 3

[6, 6, 6] rings, 5: images with three [5, 6, 7] rings, and 6: images with 3 [5, 6, 6] rings. It should be noted that only the three hexagonal structures that are attached to the central atom are considered for classification and not the orientation of the rings. For example, in an image classified as class 6, the 5 membered ring can be present in any position. Firstly, a histogram of the encoded angles of the sub-images considered for this analysis is shown in Fig. 10b. It has a characteristic twin peak similar to the case of rVAE which implies that the two rotational variants of the graphene sublayer are identified for all the classes in the dataset. Since this dataset with seven classes results in seven distinct latent spaces, they are omitted for brevity. All seven manifolds can be visualized in the jupyter notebook that accompanies this section.

For the concise presentation of latent spaces, a different way of visualizing the latent space for a dataset with discrete classes is used called the traverse manifold. In this method, all the continuous latent variables except for one are fixed and uniformly sampled images for each class is shown along this dimension. In this scenario, the continuously varied latent dimension is along the horizontal axis while the discrete class label is varied along the vertical axis. The first of these traverse manifolds is shown in Fig. 10c where z_1 is varied while the second one is shown in Fig. 10d which z_2 is varied. The angular and translational latent dimensions along with the latent dimension that is held constant are set to zero for both these manifolds. This method is similar to the decoded latent space of the rVAE where the rotational and translational encodings are set to zero. The disadvantage with this method of visualizing is that when no or few images from the dataset are encoded along the zero of the fixed dimensions, we see images that are not part of the training set in the traverse manifolds. For example, class 4 (row-5) in Fig. 10d has the ideal graphene lattices but when z_2 is fixed at zero in Fig. 10c, we see noisy images. This tells us that not much structural information is stored along $z_2 = 0$ for class 4. But these traverse manifolds give us a condensed representation of how different classes are encoded in the latent dimensions.

Further results of the crVAE analysis on the graphene dataset are shown in Fig. 11 in the real space. In Fig. 11a, the atoms considered for the analysis in the zeroth frame of the dataset are colored with their associated

Fig. 11 | Representation of ground truth labels and crVAE encodings in the image. Atoms identified by the semantic segmentation network in the zeroth frame are colored using the **a** ground truth class labels, **b** angular encodings (θ), **c** value of the first latent variable (z_1), and **d** values of the second latent variable (z_2) of the crVAE network trained on the graphene dataset.



class (ground truth). These class labels act as the input to the crVAE. In Fig. 11b, the atomic coordinates are colored with the encoded angle which shows a checkerboard type pattern similar to the one observed in rVAE. Finally, the values of the latent variables are used to color the atoms in Fig. 11c (z_1) and 11d (z_2). Not much can be concluded from these encodings without knowing the variabilities in each class of the dataset. However, Fig. 11d (z_2) also shows a checkerboard type pattern which shows that some of the information from the angular latent dimension is permeated into the second user selected latent variable z_2 . This can happen when the z_1 , z_2 latent variables encode the translational/rotational variabilities present in the dataset. For example, in this dataset, since the non-straining rotational and translational components are encoded by (θ , Δx , and Δy), and the defects are classified into different classes, the only variability that is left in the images is the strain in the structures. If z_1 encodes this, there is not much information left for the other latent variable to encode. Decreasing the number of latent dimensions in such scenarios will lead to better interpretation of the latent variables. However, the encodings by crVAE will serve as better feature vectors that encode the neighborhood of each atom for any ML analysis than the corresponding ones of rVAE since there is at least one fewer variability to encode which is provided to the network apriori. This entire analysis of crVAE on the graphene dataset can be found at ‘crvae_graphene.ipynb’ jupyter notebook that accompanies the manuscript.

Semi-supervised VAEs

Variational autoencoders are a class that belongs to the unsupervised family of machine learning algorithms. It is seldom the case that we encounter a training dataset whose data points are all divided into classes, and class labels for all the data points are known apriori. When they are known, the crVAE does an excellent job in projecting the dataset into a low dimensional latent space while explicitly disentangling the rotations and/or translations involved in the training data. The practically encountered scenario is where the data is available in the form of small, manually labeled datasets in published papers, catalogs, and other forms, and we aim to use these as partially known labels. Alternatively, the small subset of data can be labeled

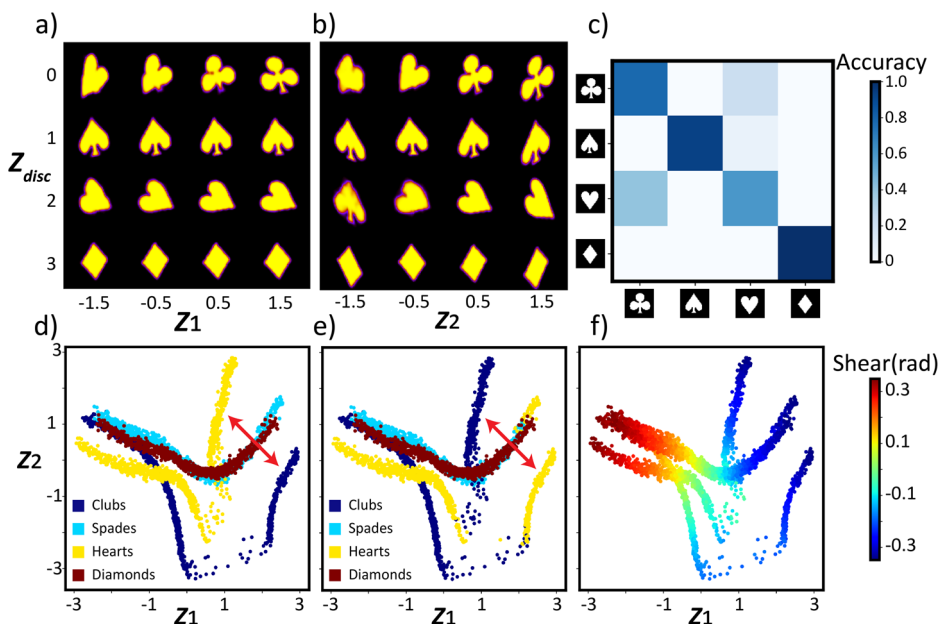
manually by human, and then we aim to propagate this label across systems with a strong rotational or translational disorder. The ssrVAE (semi-supervised rotationally invariant autoencoder) generalizes from the small, labeled subset (supervised data) to the unsupervised data that is drawn from a different distribution of variations along with a strong rotational/translational disorder. To achieve this, the ssrVAE takes advantage of important features of rVAE and crVAE. It should be noted that both crVAE and ssrVAE can also work with continuous labeled data, similar to regression tasks. In that case, we have infinite latent spaces in crVAE and ssrVAE corresponding to the continuous conditional variable.

The architecture of ssrVAE has two encoders, namely encoder-z and encoder-y. When presented with the supervised dataset: where the class labels are absent, the encoder-y acts as a classifier network which is a multi-layer perceptron with ‘softmax’ activation function where it predicts the class label for the datum, and the cross-entropy loss (classification) is used to train the encoder-y network. The encoder-z network is similar to the one we have encountered in the crVAE case, where it projects the image into low dimensional latent space while explicitly using 1–3 latent variables for disentangling the rotations and translations in the dataset. When presented with the unsupervised training data, the encoder-y predicts the class label for each data point, and this class label is concatenated to the z-vector in the latent space, and the decoding is conditioned on the class label. The difference between the unsupervised and supervised settings is that the class labels in supervised settings are used to train encoder-y, while class labels predicted by the encoder-y are used for the unsupervised case. In the unsupervised scenario, the ssrVAE aims to reconstruct the dataset by employing class labels predicted by the encoder-y network. This process capitalizes on the conditional advantages of crVAE while explicitly disentangling latent vectors associated with rotation and/or translation.

ssrVAE trained on the cards datasets

To delineate the workings of ssrVAE, we have used a small portion of the cards dataset-i (rotations = 12° , and shear = 1°) as the labeled dataset to predict the labels of the cards dataset-iii (rotations = 120° , and shear = 1°)

Fig. 12 | Results of ssrVAE trained on the cards datasets where the unsupervised dataset is drawn from cards dataset-iv and supervised dataset is drawn from cards dataset-i. Traverse manifolds where **a** first latent variable is varied (z_1) and **b** second latent variable (z_2) is varied. **c** Confusion matrix on the validation dataset. Latent distributions of the validation colored with **d** ground truth labels, **e** labels predicted by the ssrVAE network, and **f** ground truth shear values of the data.



and the cards dataset-iv (rotations = 120° , and shear = 20°). This way of choosing different distributions of datasets for supervised and unsupervised settings of ssrVAE gives us insights into how ssrVAE extrapolates the knowledge it learns from the supervised data to the unsupervised dataset. Supervised training set is constructed by choosing 200 images per card and applied transformations with the rotation and shear values randomly sampled from the cards dataset i. The unsupervised datasets comprise 3000 images per card with rotations and shear values from the cards dataset iii and iv. We trained two networks for these unsupervised datasets while holding the supervised dataset constant. The number of neurons used in the encoder- z and decoder layer for the high rotation and low shear case is 128, while for the high rotation and high shear case is 256 in each of the 2 hidden layers. Finally, to see how each network performed, two validation datasets (one for each case) with 1000 images per card suit are formed from the distributions that formed the unsupervised dataset. For brevity, the results for high rotations and high shear (cards dataset-iv) are only discussed below in this manuscript in Fig. 12.

Figure 12a, b uses the traverse manifolds to visualize the decoded latent spaces as discussed in the crVAE section. In Figure 12a, z_2 is held constant at zero while the other latent variable z_1 is varied along the x -axis and vice-versa in Fig. 12b. The discrete class label is varied along the y -axis in both the traverse manifolds. To generate these manifolds, both encoder- y and encoder- z are discarded after the training phase and class labels of each class are concatenated to the sampled z -vector to decode the latent vectors. It can be observed that the class 0 (clubs, first row) and class 2 (hearts, third row) have some misclassifications. This can also be inferred from the normalized confusion matrix shown in Fig. 12c. The accuracy rates of clubs and spades is around 50% with the images being identified as each other. The latent distributions of the validation dataset are plotted in Fig. 12d, e and colored using the ground truth class labels in 12d and the predicted class labels in 12e.

The misclassification between clubs and spades are shown using red arrows in Fig. 12d, e where the two branches in the latent distribution seem to have exchanged classes. This is due to the large difference in distribution between the supervised and unsupervised datasets and the structural similarities between the highly sheared hearts and the clubs classes (can also be visualized from the traverse manifolds). Finally, the latent distributions of the validation dataset are colored using ground truth shear values in Fig. 12f which shows a smooth variation of shear in the latent space. Even though the latent distributions of shear values of different classes seem to have overlapped, as discussed in the crVAE case, each class lives in a distinct latent

space. The latent space in which each of these encodings live is a function of the predicted class label by encoder- y which is denoted by the color in Fig. 12e. It should be noted that in the supervised training dataset, the maximum value of shear present is 1° and the ssrVAE generalized it to the unsupervised dataset where the maximum value of shear is 20° . The trained ssrVAE has achieved an accuracy of 82% on the unsupervised dataset by generalizing the features learnt from the supervised dataset. In conclusion, ssrVAE, when presented with 800 images with low rotations (12°) and low shear (1°), is able to classify with 82% accuracy, explicitly disentangle the rotations, and form a smooth varying latent space of other variations of the high rotation (120°) and high shear (20°) dataset with 12,000 images. The complete analysis of the two datasets can be accessed and visualized using the 'ssrvaecards.ipynb' jupyter notebook that accompanies the manuscript.

ssrVAE trained on the experimental dataset

Graphene dataset setting discussed in the crVAE section is used to illustrate the workings of ssrVAE as well. Only 15% of the input images that fall into one of the seven classes discussed in the crVAE section are used as the supervised dataset. A small portion (4%) of the dataset is set aside as the validation dataset. The remaining dataset including the images that do not fall into the seven categories discussed in the crVAE section is used as the unsupervised dataset. The traversal manifold way visualization is shown in Fig. 13a, b. The first latent variable (z_1) is varied in Fig. 13a while the z_2 is held constant at zero and vice-versa in Fig. 13b. These manifolds provide insight into how different classes are encoded in the latent dimensions $z_1 - z_2$. The confusion matrix on the validation dataset is shown in Fig. 13c. The network is able to predict the class-4 i.e., the ideal [6, 6, 6] with the highest accuracy. The final accuracy on the validation dataset is 74.6%.

Further results of the latent encodings are shown in Fig. 14. To obtain the encodings, the whole dataset is used as the input and each atom in the zeroth frame is colored using the encoded angle in Fig. 14a. The checkerboard pattern observed in the previous sections is prevalent in this scenario as well. The latent dimensions z_1 and z_2 are used to color Fig. 14b, c, respectively. Since the ssrVAE is conditioned on the class predicted by encoder- y , these encoding values reside in distinct latent spaces. This means that the structural variability in the neighborhoods is already encoded by encoder- y . Similar to the latent dimensions the case of crVAE, they encode the strain and microscopic variabilities which can be observed from the traversal manifolds in Fig. 13a, b. Finally, the atoms are colored with the classes predicted by encoder- y in Fig. 14d. A large portion of the frame is

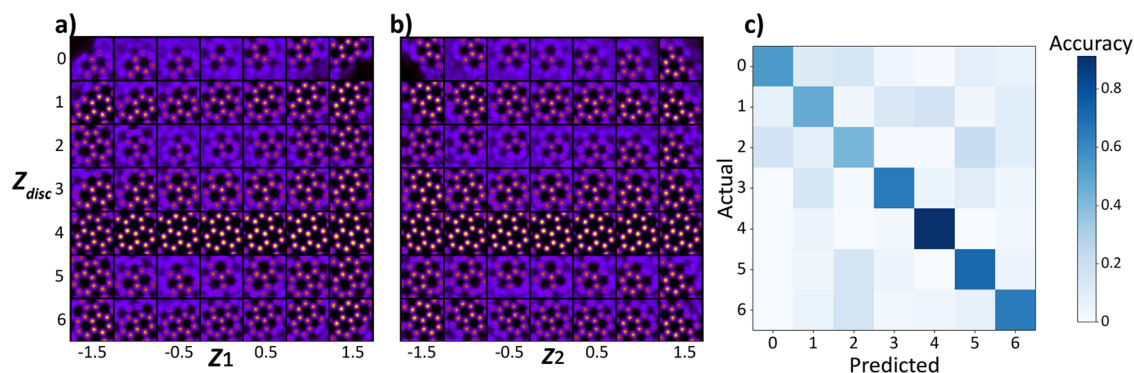
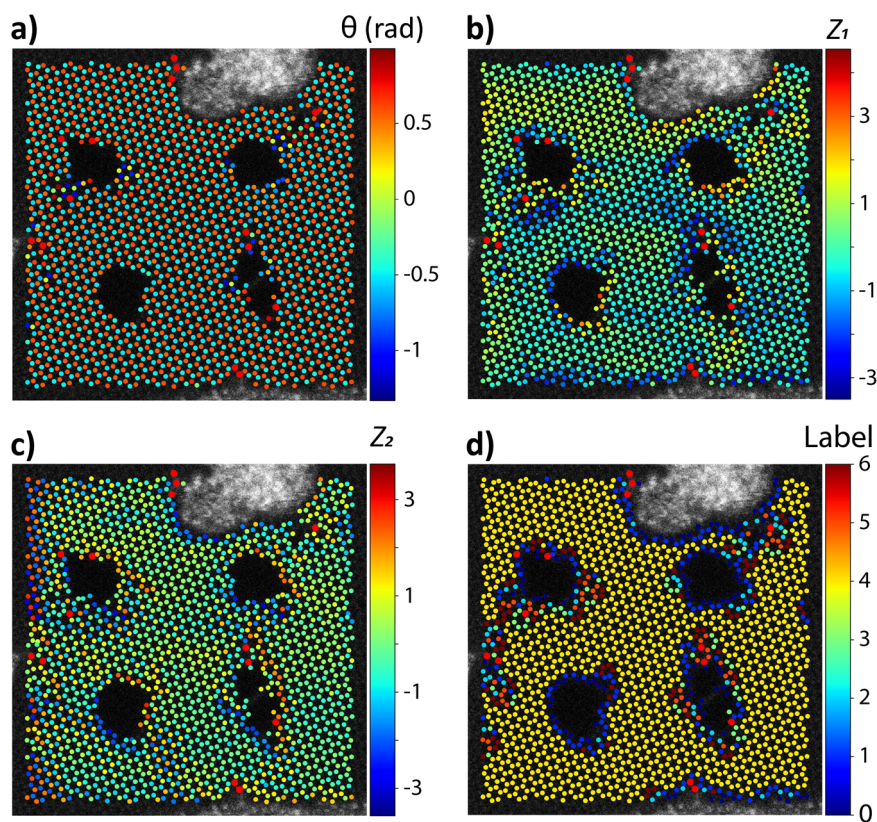


Fig. 13 | Results of ssrVAE trained on the graphene dataset. Traverse manifolds when **a** first latent variable (z_1) is varied and **b** second latent variable (z_2) is varied. **c** Confusion matrix of the validation dataset of graphene in the ssrVAE setting.

Fig. 14 | Representation of ssrVAE encodings and predicted labels in the image. Atoms identified by the semantic segmentation network in the zeroth frame are colored using the **a** angular encodings (θ), **b** value of the first latent variable (z_1), and **c** values of the second latent variable (z_2), **d** class labels predicted by the ssrVAE network trained on the graphene dataset.



occupied by the ideal graphene lattice class 4, the class that contains the majority of input datapoints.

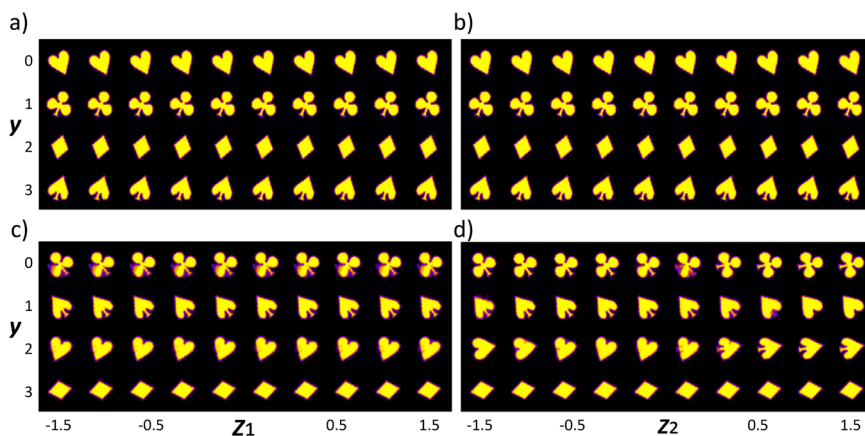
Joint VAEs

Generally, the imaging data comprises objects from multiple classes with variability in images within the class. In supervised and semi-supervised machine learning settings, at least a part of the dataset is labeled. The algorithm then propagates the information learned from the known classes to the unlabeled dataset. However, the classes in the dataset are not apparent, especially in disciplines where a lot of labeled datasets are not readily available. Unsupervised clustering techniques divide the dataset into discrete classes based on the input dataset's hidden patterns or data groupings. The problem of learning low-dimensional manifolds while dividing an unlabeled dataset into discrete classes poses an ill-suited one for traditional machine algorithms.

This section discusses a class of invariant VAEs called the joint rotationally invariant variational autoencoders (jrVAEs) suited for solving this problem. The 'joint' keyword in the name describes the aspect of the latent space of jrVAE where the latent space of the jrVAE jointly comprises discrete and continuous latent dimensions. This is similar to the unsupervised setting of the ssrVAE with the untrained classifier network. The discrete latent dimensions of the jrVAE are responsible for disentangling the dataset into various classes, while the continuous dimensions learn the in-class variations. These aspects can also be independently combined with the rotational and translational invariance, where 1-3 latent dimensions capture the rotations and translations involved in the images.

The ELBO for the jrVAE is described by the Eq. 5 below comprising of three terms, where the first term is the reconstruction error refers to the error between the original and reconstructed image. The second term has three different components corresponding to the Kullback-Leibler (KL)

Fig. 15 | Results of jrVAE trained on the cards datasets. Traverse manifolds of jrVAE trained on cards dataset-i when **a** z_1 is varied, **b** z_2 is varied, and on cards dataset iii when **c** z_1 is varied, **d** z_2 is varied.



divergence between the continuous latent variables and their corresponding priors. The first component $D_{KL}(q(z, |, x) \mathcal{N}(0, I))$ is the KL divergence between the latent variables (z) and standard Normal prior distribution. This is followed by $D_{KL}(q(\theta|x) || \mathcal{N}(0, \sigma_\theta^2))$ which is the KL divergence between the encodings of the angular dimension (θ) and its corresponding Normal prior with zero mean and standard deviation σ_θ . The last component of the second term $D_{KL}(q(\Delta x, |, x) \mathcal{N}(0, \sigma_s^2))$ refers to the KL divergence between the translational dimension and its normal prior with a standard deviation σ_s . Finally, the last term is the penalty on the discrete dimensions and is represented as the KL-divergence between the discrete latent variable and its corresponding prior $p(y) = \text{Cat}(y|\lambda)$ which is a discrete categorical distribution with parameters λ .

$$\begin{aligned} ELBO_{jrvae} = & -\text{Reconstruction error} \\ & -\gamma_c |D_{KL}(q(z|x) \mathcal{N}(0, I)) + D_{KL}(q(\theta|x) \mathcal{N}(0, \sigma_\theta^2)) \\ & + D_{KL}(q(\Delta x|x) || \mathcal{N}(0, \sigma_s^2)) - C_c| \\ & -\gamma_{disc} |D_{KL}(q(y|x) || p(y)) - C_{disc}| \end{aligned} \quad (5)$$

The hyperparameters of the ELBO that are to be optimized for a given dataset involve σ_θ : parameter in the prior on θ , σ_s : standard deviation of the Normal prior on Δx , λ : parameters of the categorical prior distribution on the discrete latent variables. Along with these, the jrVAE network is highly sensitive to there are three other hyperparameters viz., γ , C_c and C_{disc} . The continuous and discrete channel capacities are C_c and C_{disc} respectively. When these are set to low values at a constant value of the weight coefficient γ , the effect of the KL terms diminishes and leads to a higher emphasis on the reconstruction error. The effect of these three terms is identical to that of parameter β in the β -VAEs. During the initial steps of the training a low value of the channel capacities are used for a better reconstruction of the data initially and then it is increased during training for the latent distributions to match the priors. It should be thoroughly noted that although other variants of VAEs (mainly ssrVAE) are weakly dependent on these hyperparameters, the efficacy of jrVAEs, in our experience depends heavily on optimizing them.

jrVAE trained on the cards datasets

With the concise introduction to jrVAEs, in this section the cards datasets are used to delineate their capabilities. One jrVAE network for each of the four datasets are trained where 1024 neurons are used in hidden layers. A total of four hidden layers are used in both the encoder and decoder networks. Two user selected latent variables (z_1 , z_2), three invariant latent variables (θ , Δx , and Δy) and one discrete latent variable (y) with 4 dimensions comprise the latent space. The class label predicted by the jrVAE is the dimension of discrete latent variable with the highest value. The results of jrVAEs on cards datasets i and iii are discussed here. However, the results of all four datasets can be found in the jupyter notebook that accompanies

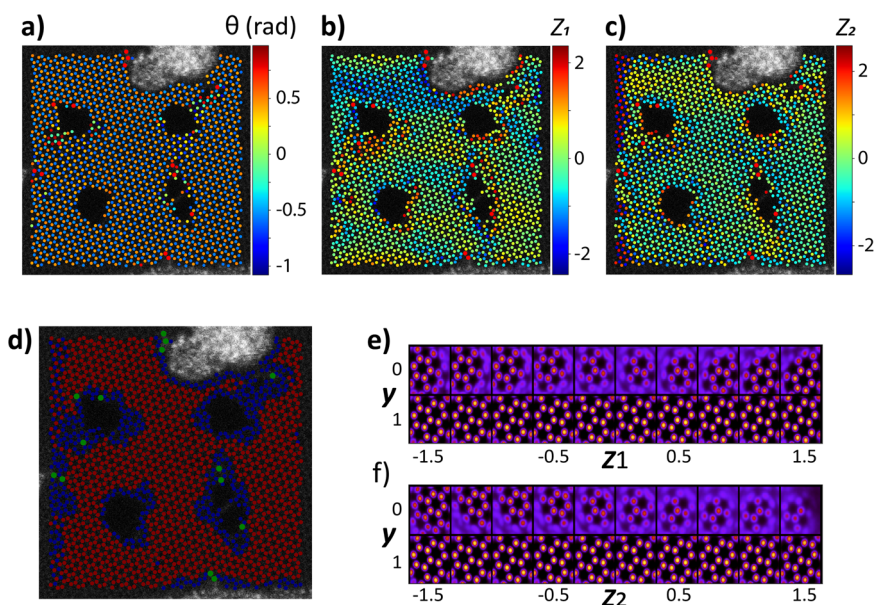
this section. The traverse manifolds for cards dataset i are shown in Fig. 15a, b, where z_1 is varied in Fig. 15a and z_2 is varied in Fig. 15b while the rest of the continuous variables are held constant at zero. The traverse manifolds show that for the case of cards dataset i, jrVAE does an optimal job in identifying four classes without a prior knowledge about classes. The four latent spaces, latent distributions, and angle plots can all be visualized in the jupyter notebook. The traverse manifolds for cards dataset iii are shown in Fig. 15c (z_1 is varied) and 15d (z_2 is varied). The jrVAE in this case does an ideal job for diamonds as the class corresponds to diamonds (class 3) is filled with all diamonds suit in a single orientation. The network also identified all the clubs as a single class (class 0) albeit with multiple orientations. We have observed this trend of having multiple ground state zeros for the angular latent dimension in other variants of VAEs when the rotations in the dataset are high. Finally, both spades and hearts suits occupy both classes 1 and 3 due to the structural similarities present between them. We can also see that the hearts are identified with two different ground state orientations. Although, the classes predicted by the jrVAE arbitrary, each of the four classes is given a class label based on the majority ground truth classes. Using this predicted class labels, the accuracy of the jrVAE on the cards dataset iii is 82.11%. This calculation of accuracy is possible because we knew the ground truth class labels apriori. The complete analyses of all four datasets, latent spaces of all classes, latent distributions, angle plots, and shear variation in the latent space plots can be found in the 'jrVAE_card.ipynb' jupyter notebook that accompanies the manuscript.

As noted earlier, the jrVAEs are highly sensitive to hyperparameters γ , C_z , and C_y . These hyperparameters should be optimized for each dataset to produce ideal results. In this article, we have used the default parameters that come with the AtomAI package used to create the jrVAE network. Optimizing these hyperparameters is beyond the scope of the review and an elegant way to optimize them for each dataset is proposed in ref. 58 The default parameters often result in sub-standard performance and can only be used as an initial step in understanding the dataset.

6.b. jrVAE trained on the experimental dataset

The results of the application of jrVAE on the graphene dataset are discussed in Fig. 16. The network architecture used for this application is similar to that of the cards dataset with the only change in dimensionality of the discrete latent dimension (y) which is set to 2 for this case. The encodings in the angular dimension, z_1 , and z_2 are shown on top of the detected atomic coordinates of zeroth frame in Fig. 16a–c, respectively. The checkerboard pattern in the angular latent dimension is also observed for jrVAE case as well which suggests that the jrVAE also found an optimal encoding for the angle. For the encodings of z_1 and z_2 , since each sub-image resides in either of the two independent latent spaces, these have to be considered along with the class label to understand the variability along these dimensions. The predicted class label is used to color the atomic coordinates in Fig. 16d, where large green dots correspond to the detected Si atoms. The traverse manifolds

Fig. 16 | Results of jrVAE trained on the graphene dataset. Atoms identified by the semantic segmentation network in the zeroth frame are colored using the **a** angular encodings (θ), **b** value of the first latent variable (z_1), and **c** value of the second latent variable (z_2), and **d** class labels (y) predicted by the jrVAE network trained on the graphene dataset. Traverse manifolds of the jrVAE when z_1 is varied and **e** z_2 is varied.



are shown in Fig. 16e (z_1 is varied) and Fig. 16f (z_2 is varied). Figure 16d–f shows that the zeroth class (red dots in Fig. 16d) corresponds to the ideal graphene lattice while the first class (blue dots in Fig. 16e) corresponds to the defects and closeness to the amorphous regions. Conditioned on these classes, the latent variables z_1 and z_2 encode the additional variabilities in the feature vectors. The entire analysis can be reproduced and visualized in ‘jrvaes_graphene.ipynb’ jupyter notebook that accompanies the manuscript.

Discussion

To summarize, here we discuss the basic principles, implementations, and applications of the invariant variational autoencoders for the imaging data. The central concept of VAE is the disentanglement of the representations of the data, i.e., finding the parsimonious factors of variability. In the classical datasets like MNIST, the factors of variability refer to the characteristics of handwriting style such as tilt and width of letters. In imaging data, factors of variability include shape variations of nanoparticles, distortions of the unit cells due to strain or ferroelectric polarization, chemical neighborhoods of atoms in graphene, or variability of the local diffraction patterns in 4D STEM. Correspondingly, VAEs offer a potential pathway to learn these behaviors from observational data.

However, the direct applications of the simple VAEs to the experimental data is limited. First and foremost, in observational studies multiple objects of the same type can be observed at different observation angles. Second, the sampling procedure to identify the sub-images can be associated with the random shifts. Finally, the system can be associated with the presence of affine distortions, that can stem from intrinsic material structure, or be due to the imaging system. The shift- and rotation invariant VAEs introduced here allow separating these physical factors of variability directly and disentangle the remaining factors of variability in latent representations.

Similarly, very often the imaging data is collected from system with multiple discrete object classes, that can either be known, partially known (have examples), or unknown. We discuss conditional, semi-supervised, and joint VAEs as frameworks to address these tasks. Notably, conditional VAEs can also be extended towards the continuous conditional vectors, allowing to incorporate known physical factors of variability.

Jointly, this framework allows comprehensive analysis of imaging datasets. Some of the intuition beyond the formation and properties of the latent spaces and latent manifolds is discussed. Finally, we note that VAE approach can be extended to more complex tasks of building structure property relations and active learning but defer these to future publications.

The notebooks illustrating the concepts in this review are fully available <https://github.com/saimani5/VAE-tutorials>. We encourage the readers to explore these concepts further and apply VAE to own data.

Data availability

The jupyter notebooks containing the datasets and code to completely reproduce the analyses discussed in the article are available for public use under the GitHub repository: <https://github.com/saimani5/VAE-tutorials>. The datasets used in each section are downloaded into the notebooks before the analyses. Readers can run the jupyter notebooks directly in a Google Colab environment to reproduce the results.

Code availability

The codes that can be used to completely reproduce the analyses discussed in this work are available at <https://github.com/saimani5/VAE-tutorials> as a repository of jupyter notebooks.

Received: 15 August 2023; Accepted: 22 March 2024;

Published online: 14 August 2024

References

- van Zuylen, J. The microscopes of Antoni van Leeuwenhoek. *J. Microsc.* **121**, 309–328 (1981).
- Ntziachristos, V. Going deeper than microscopy: the optical imaging frontier in biology. *Nat. Methods* **7**, 603–614 (2010).
- Fan, Z. et al. In situ transmission electron microscopy for energy materials and devices. *Adv. Mater.* **31**, 1900608 (2019).
- Bruma, A. *Scanning Transmission Electron Microscopy: Advanced Characterization Methods for Materials Science Applications* (CRC Press, 2020).
- Wang, Y., Skaavvik, S. A., Xiong, X., Wang, S. & Dong, M. Scanning probe microscopy for electrocatalysis. *Matter* **4**, 3483–3514 (2021).
- Wen, H., Cherukara, M. J. & Holt, M. V. Time-resolved X-ray microscopy for materials science. *Annu. Rev. Mater. Res.* **49**, 389–415 (2019).
- Hui, F. & Lanza, M. Scanning probe microscopy for advanced nanoelectronics. *Nat. Electron.* **2**, 221–229 (2019).
- Gerber, C. & Lang, H. P. How the doors to the nanoworld were opened. *Nat. Nanotechnol.* **1**, 3–5 (2006).
- Binnig, G. & Rohrer, H. Scanning tunneling microscopy. *Helvetica Phys. Acta* **55**, 726–735 (1982).

10. Strosio, J. A., Feenstra, R. M. & Fein, A. P. Electronic-structure of the Si(111)2x1 surface by scanning-tunneling microscopy. *Phys. Rev. Lett.* **57**, 2579–2582 (1986).
11. Asenjo, A., Gomezrodriguez, J. M. & Baro, A. M. Current imaging tunneling spectroscopy of metallic deposits on silicon. *Ultramicroscopy* **42**, 933–939 (1992).
12. Grutter, P., Liu, Y., LeBlanc, P. & Durig, U. Magnetic dissipation force microscopy. *Appl. Phys. Lett.* **71**, 279–281 (1997).
13. Martin, Y. & Wickramasinghe, H. K. Magnetic imaging by force microscopy with 1000-Å resolution. *Appl. Phys. Lett.* **50**, 1455–1457 (1987).
14. Noy, A., Vezennov, D. V. & Lieber, C. M. Chemical force microscopy. *Annu. Rev. Mater. Sci.* **27**, 381–421 (1997).
15. Gruverman, A., Auciello, O. & Tokumoto, H. Imaging and control of domain structures in ferroelectric thin films via scanning force microscopy. *Annu. Rev. Mater. Sci.* **28**, 101–123 (1998).
16. Kim, M. et al. Nonstoichiometry and the electrical activity of grain boundaries in SrTiO₃. *Phys. Rev. Lett.* **86**, 4056–4059 (2001).
17. Pennycook, S. J., Varela, M., Lupini, A. R., Oxley, M. P. & Chisholm, M. F. Atomic-resolution spectroscopic imaging: past, present and future. *J. Electron Microsc.* **58**, 87–97 (2009).
18. Sohlberg, K., Rashkeev, S., Borisevich, A. Y., Pennycook, S. J. & Pantelides, S. T. Origin of anomalous Pt–Pt distances in the Pt/alumina catalytic system. *Chemphyschem* **5**, 1893–1897 (2004).
19. Chisholm, M. F., Luo, W. D., Oxley, M. P., Pantelides, S. T. & Lee, H. N. Atomic-scale compensation phenomena at polar interfaces. *Phys. Rev. Lett.* **105**, 197602 (2010).
20. Browning, N. D. et al. The influence of atomic structure on the formation of electrical barriers at grain boundaries in SrTiO₃. *Appl. Phys. Lett.* **74**, 2638–2640 (1999).
21. Clausen-Schaumann, H., Seitz, M., Krautbauer, R. & Gaub, H. E. Force spectroscopy with single bio-molecules. *Curr. Opin. Chem. Biol.* **4**, 524–530 (2000).
22. Rief, M., Oesterhelt, F., Heymann, B. & Gaub, H. E. Single molecule force spectroscopy on polysaccharides by atomic force microscopy. *Science* **275**, 1295–1297 (1997).
23. Balke, N., Bdiqin, I., Kalinin, S. V. & Kholkin, A. L. Electromechanical imaging and spectroscopy of ferroelectric and piezoelectric materials: state of the art and prospects for the future. *J. Am. Ceram. Soc.* **92**, 1629–1647 (2009).
24. Van der Walt, S. et al. scikit-image: image processing in Python. *PeerJ* **2**, e453 (2014).
25. Krizhevsky, A., Sutskever, I. & Hinton, G. E. Imagenet classification with deep convolutional neural networks. *Commun. ACM* **60**, 84–90 (2017).
26. Opitz, D. & MacLain, R. Popular ensemble methods: an empirical study. *J. Artif. Intell. Res.* **11**, 169–198 (1999).
27. Vyas, A., Jammalamadaka, N., Zhu, X., Das, D., Kaul, B. & Willke, T. L. Out-of-Distribution Detection Using an Ensemble of Self Supervised Leave-Out Classifiers. *Computer Vision – ECCV 2018*, 560–574 (Springer International Publishing, Cham 2018).
28. Xia, G. & Bouganis, C.-S. On the usefulness of deep ensemble diversity for out-of-distribution detection. Preprint at <https://arxiv.org/abs/2207.07517> (2022).
29. Liu, Y., Kelley, K. P., Funakubo, H., Kalinin, S. V. & Ziatdinov, M. Exploring physics of ferroelectric domain walls in real time: deep learning enabled scanning probe microscopy. *Adv. Sci.* **9**, 2203957 (2022).
30. Liu, Y. et al. Disentangling electronic transport and hysteresis at individual grain boundaries in hybrid perovskites via automated scanning probe microscopy. *ACS Nano* **17**, 9647–9657 (2023).
31. Kingma, D. P. & Welling, M. Auto-Encoding Variational Bayes. 2nd International Conference on Learning Representations, ICLR, Banff, AB, Canada (2014).
32. Kingma, D. P. & Welling, M. An introduction to variational autoencoders. *Found. Trends® Mach. Learn.* **12**, 307–392 (2019).
33. Rezende, D. J., Mohamed, S. & Wierstra, D. In *International conference on machine learning*. 1278–1286 (PMLR).
34. Taud, H. & Mas, J. Multilayer perceptron (MLP). *Geomatic approaches for modeling land change scenarios* (eds Olmedo MTC, Paegelow M, Mas J-F, Escobar F) 451–455 (Springer International Publishing, Cham, 2018).
35. Goodfellow, I., Bengio, Y. & Courville, A. Convolutional networks Vol. 2016 *Deep learning* 330–372 (MIT Press Cambridge, 2016).
36. Vaswani, A. et al. Attention is all you need. In *Proc. of the 31st International Conference on Neural Information Processing Systems* 6000–6010 (Long Beach, California, USA, Curran Associates Inc, 2017).
37. de Haan, P., Cohen, T. S. & Welling, M. Natural graph networks. *Adv. Neural Inf. Process. Syst.* **33**, 3636–3646 (2020).
38. Kalinin, S. V., Kelley, K., Vasudevan, R. K. & Ziatdinov, M. Toward decoding the relationship between domain structure and functionality in ferroelectrics via hidden latent variables. *ACS Appl. Mater. Interfaces* **13**, 1693–1703 (2021).
39. Roccapiore, K. M., Ziatdinov, M., Cho, S. H., Hachtel, J. A. & Kalinin, S. V. Predictability of localized plasmonic responses in nanoparticle assemblies. *Small* **17**, 2100181 (2021).
40. Ziatdinov, M., Ghosh, A., Wong, C. Y. & Kalinin, S. V. AtomAI framework for deep learning analysis of image and spectroscopy data in electron and scanning probe microscopy. *Nat. Mach. Intell.* **4**, 1101–1112 (2022).
41. Lecun, Y., Bottou, L., Bengio, Y. & Haffner, P. Gradient-based learning applied to document recognition. *Proc. IEEE* **86**, 2278–2324 (1998).
42. Duvenaud, D. K. et al. Convolutional networks on graphs for learning molecular fingerprints. *Adv. Neural Inf. Process. Syst.* **28**, 2224–2232 (2015).
43. Weininger, D. SMILES, a chemical language and information system. 1. Introduction to methodology and encoding rules. *J. Chem. Inf. Comput. Sci.* **28**, 31–36 (1988).
44. Krenn, M., Häse, F., Nigam, A., Friederich, P. & Aspuru-Guzik, A. Self-referencing embedded strings (SELFIES): A 100% robust molecular string representation. *Mach. Learn. Sci. Technol.* **1**, 045024 (2020).
45. Kalinin, S. V., Steffes, J. J., Liu, Y., Huey, B. D. & Ziatdinov, M. Disentangling ferroelectric domain wall geometries and pathways in dynamic piezoresponse force microscopy via unsupervised machine learning. *Nanotechnology* **33**, 055707 (2021).
46. Liu, Y., Ziatdinov, M. & Kalinin, S. V. Exploring causal physical mechanisms via non-gaussian linear models and deep kernel learning: applications for ferroelectric domain structures. *ACS Nano* **16**, 1250–1259 (2021).
47. Valleti, S. M. P., Ignatans, R., Kalinin, S. V. & Tileli, V. Decoding the mechanisms of phase transitions from in situ microscopy observations. *Small* **18**, 2104318 (2022).
48. Liu, Y. et al. Decoding the shift-invariant data: applications for band-excitation scanning probe microscopy. *Mach. Learn. Sci. Technol.* **2**, 045028 (2021).
49. Liu, Y., Proksch, R., Wong, C. Y., Ziatdinov, M. & Kalinin, S. V. Disentangling ferroelectric wall dynamics and identification of pinning mechanisms via deep learning. *Adv. Mater.* **33**, 2103680 (2021).
50. Liu, Y., Huey, B. D., Ziatdinov, M. A. & Kalinin, S. V. Physical discovery in representation learning via conditioning on prior knowledge: applications for ferroelectric domain dynamics. Preprint at <https://arxiv.org/abs/2203.03122> (2022).
51. Bepler, T., Zhong, E., Kelley, K., Brignole, E. & Berger, B. Explicitly disentangling image content from translation and rotation with spatial-VAE. *Adv. Neural Inf. Process. Syst.* **32**, 15409–15419 (2019).
52. Kalinin, S. V., Dyck, O., Jesse, S. & Ziatdinov, M. Exploring order parameters and dynamic processes in disordered systems via variational autoencoders. *Sci. Adv.* **7**, eabd5084 (2021).

53. Ziatdinov, M. et al. Building and exploring libraries of atomic defects in graphene: Scanning transmission electron and scanning tunneling microscopy study. *Sci. Adv.* **5**, eaaw8989 (2019).
54. Dyck, O., Kim, S., Kalinin, S. V. & Jesse, S. Placing single atoms in graphene with a scanning transmission electron microscope. *Appl. Phys. Lett.* **111**, 113104 (2017).
55. Dyck, O., Kim, S., Kalinin, S. V. & Jesse, S. E-beam manipulation of Si atoms on graphene edges with an aberration-corrected scanning transmission electron microscope. *Nano Res.* **11**, 6217–6226 (2018).
56. Dyck, O. et al. Building structures atom by atom via electron beam manipulation. *Small* **14**, 1801771 (2018).
57. Ziatdinov, M. et al. Deep learning of atomically resolved scanning transmission electron microscopy images: chemical identification and tracking local transformations. *ACS Nano* **11**, 12742–12752 (2017).
58. Biswas, A., Vasudevan, R., Ziatdinov, M. & Kalinin, S. V. Optimizing training trajectories in variational autoencoders via latent Bayesian optimization approach. *Mach. Learn. Sci. Technol.* **4**, 015011 (2023).

Acknowledgements

This work (workflow development, manuscript writing) was supported by the US Department of Energy, Office of Science, Office of Basic Energy Sciences, as part of the Energy Frontier Research Centers program: CSSAS — The Center for the Science of Synthesis Across Scales — under Award No. DE-SC0019288, located at University of Washington, DC. Additional support for ongoing pyroVED software development came from the Laboratory Directed Research and Development Program at Pacific Northwest National Laboratory (PNNL), a multiprogram national laboratory operated by Battelle for the U.S. Department of Energy. This research was also partially supported by the Center for Nanophase Materials Sciences (CNMS), which is a US Department of Energy, Office of Science User Facility at Oak Ridge National Laboratory (ORNL).

Author contributions

Mani Valleti: developed the jupyter notebooks responsible for generating the results presented in the paper, conducted data analysis, and contributed to the writing of manuscript. Maxim Ziatdinov: created the Python packages

‘atomai’ and ‘pyroved’, which were instrumental in constructing the Variational Autoencoder (VAE) networks central to the research. Also participated in the analysis of the results. Yongtao Liu: assisted with the result analysis and contributed towards the writing of manuscript. Sergei Kalinin: originated the research concept, provided oversight throughout the project’s development, assisted with result analysis, and contributed to the manuscript’s composition.

Competing interests

The authors declare no competing interests.

Additional information

Correspondence and requests for materials should be addressed to Mani Valleti or Sergei V. Kalinin.

Reprints and permissions information is available at <http://www.nature.com/reprints>

Publisher’s note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article’s Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article’s Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

© UT-Battelle, LLC, Battelle Memorial Institute and Mani Valleti 2024